ארכיטקטורת תוכנה לסביבת סוכני AI

Software Architecture for AI Agent Environment Based on MCP Servers

ד"ר יורם סגל

© Dr. Segal Yoram - כל הזכויות שמורות

December 2025

גרסה 1.0 - דצמבר 2025

תוכן העניינים

1 מבוא: עידן הסוכנים החכמים 7

1.1 פתיחה: המהפכה בעולם הבינה המלאכותית 7

1.2 מה הקשר בין סוכנים לבין MCP? 7

1.3 למי מיועד הספר הזה? 7

1.4 חדשות מעולם ה-AI: סוף שנת 2025 8

1.5 הפרדה בין נתונים לתצוגה: העיקרון המנחה 9

1.6 מבנה הספר 9

1.7 איך לקרוא את הספר? 9

2 ארכיטקטורת שרת-לקוח בעולם ה-MCP 11

2.1 מבוא: למה אנחנו צריכים ארכיטקטורה? 11

2.2 ארכיטקטורת שרת-לקוח: העיקרון הבסיסי 11

2.3 המקרה המיוחד של MCP: תפקידים דינמיים 12

2.4 שלושת השחקנים ב-MCP 12

2.5 תרשים ארכיטקטורה 13

2.6 קוד לדוגמה: הגדרת שרת MCP פשוט 14

2.7 זרימת התקשורת 15

2.8 יתרונות הארכיטקטורה 15

2.9 אתגרים ופתרונות 15

2.10 סיכום הפרק 16

3 שלושת הפרימיטיבים של MCP 17

3.1 מבוא: שפת השיחה בין סוכנים 17

3.2 Tools — כלים: היכולת לפעול 17

3.3 Resources — משאבים: היכולת לקרוא 19

3.4 Prompts — תבניות: היכולת להנחות 21

3.5 השוואה בין שלושת הפרימיטיבים 24

3.6 תרשים זרימה: שימוש בפרימיטיבים 24
3.7 שילוב הפרימיטיבים בפועל 24
3.8 סיכום הפרק 26

4 ארכיטקטורת תוכנה לסוכני AI 27
4.1 מבוא: למה ארכיטקטורה חשובה כל כך? 27
4.2 ארכיטקטורה שכבתית: Clean Architecture 27
4.3 כלל הזהב: מקסימום 150 שורות לקובץ 29
4.4 דפוס Repository: הפרדת נתונים מלוגיקה 31
4.5 הזרקת תלויות: Dependency Injection 34
4.6 ניהול משאבים בסביבה רב-תהליכית 37
4.7 תרשים ארכיטקטורה מלא 39
4.8 סיכום הפרק 39

5 שכבת התקשורת 41
5.1 מבוא: איך הודעות עוברות? 41
5.2 שתי שכבות התקשורת 41
5.3 STDIO: תקשורת מקומית 41
5.4 HTTP/SSE: תקשורת ברשת 44
5.5 סנכרון ונעילות 46
5.6 הגבלת קצב: Rate Limiting 48
5.7 Watchdog: זיהוי תקיעות 50
5.8 סיכום הפרק 52

6 SDK וממשק משתמש 53
6.1 מבוא: למה להפריד SDK מ-GUI? 53
6.2 מה זה SDK? 53
6.3 ארכיטקטורת SDK 53
6.4 מודלים: Pydantic 56
6.5 ממשקי משתמש מרובים 58
6.6 דרישות ממשק משתמש 60
6.7 סיכום הפרק 65

7 בדיקות יחידה והבטחת איכות 67
7.1 מבוא: למה בדיקות כל כך חשובות? 67
7.2 סוגי בדיקות 67

7.3 Unit Tests: היסוד 67
7.4 מוקים: בידוד תלויות 69
7.5 בדיקות פרמטריות 71
7.6 בדיקות אינטגרציה 72
7.7 ארגון תיקיית הבדיקות 74
7.8 כיסוי קוד 75
7.9 פיתוח מונחה בדיקות: TDD 75
7.10 סיכום הפרק 78

8 יישום מעשי: מערכת חשבוניות 79
8.1 מבוא: מתיאוריה למעשה 79
8.2 שלב 1: תשתית בסיסית 79
8.3 שלב 2: הוספת כלים 82
8.4 שלב 3: הוספת משאבים 85
8.5 שלב 4: SDK וממשקים 86
8.6 שלב 5: בדיקות 91
8.7 סיכום הפרויקט 92
8.8 צעדים הבאים 92
8.9 סיכום הספר 93

נספח א: פיתוח מקבילי עם Git Worktrees 95
8.10 פתיחה: הבעיה העתיקה של עבודה במקביל 95
8.11 הפתרון: Git Worktrees 95
8.12 הדפוס של Anthropic: סוכן לכל ענף 96
8.13 מדריך מעשי: כיצד להפעיל צוות סוכנים 96
8.14 ניהול סשנים: כוחו של הזיכרון 99
8.15 דוגמה מהעולם האמיתי: פיתוח מערכת חשבוניות 100
8.16 טיפים מתקדמים 100
8.17 סיכום: העתיד כבר כאן 101
8.18 מקורות בעברית 102
8.19 English References 102

כל הזכויות שמורות - © Dr. Segal Yoram

1 מבוא: עידן הסוכנים החכמים

1.1 פתיחה: המהפכה בעולם הבינה המלאכותית

בעולם הטכנולוגיה של ימינו, אנו עדים למהפכה שקטה אך עמוקה. לא מדובר עוד במודלי שפה גדולים (Large Language Models - LLMs) שעונים על שאלות בלבד, אלא בסוכנים חכמים (AI Agents) המסוגלים לפעול באופן עצמאי, לתקשר עם מערכות חיצוניות, ולבצע משימות מורכבות בשמנו.

דמיינו לרגע: במקום לכתוב קוד שורה אחר שורה, אתם מתארים למחשב מה אתם רוצים להשיג, והוא בונה את התוכנה בעצמו. במקום לחפש מידע באינטרנט ולסכם אותו, סוכן חכם עושה זאת עבורכם ומציג לכם את התמצית. זו לא עוד חזון עתידני — זו המציאות של היום.

1.2 מה הקשר בין סוכנים לבין MCP?

בלב ההתפתחות הזו עומד פרוטוקול חדש יחסית: Model Context Protocol (MCP). פרוטוקול זה, שפותח על ידי Anthropic, מגדיר את השפה המשותפת שבה סוכנים חכמים מתקשרים עם העולם החיצון.

כשם שפרוטוקול HTTP אפשר לדפדפנים לתקשר עם שרתים ויצר את רשת האינטרנט כפי שאנו מכירים אותה, כך MCP מאפשר לסוכני AI לתקשר עם כלים, משאבים ומערכות חיצוניות. זוהי התשתית שעליה נבנה העתיד של האוטומציה החכמה.

1.3 למי מיועד הספר הזה?

ספר זה מיועד למפתחים, מהנדסי תוכנה, וכל מי שמעוניין להבין כיצד לבנות מערכות מבוססות סוכני AI. אין צורך בניסיון קודם עם MCP, אך ידע בסיסי בתכנות ובעקרונות ארכיטקטורת תוכנה יסייע רבות.

1.3.1 מה תלמדו בספר זה?

- **עקרונות ארכיטקטורה** — כיצד לתכנן מערכת מודולרית שתומכת במאות ואלפי סוכנים
- **פרוטוקול MCP** — הפרימיטיבים הבסיסיים: Tools, Resources, ו-Prompts
- **שכבת תקשורת** — STDIO לעומת HTTP/SSE ומתי להשתמש בכל אחד
- **SDK וממשקים** — כיצד להפריד בין לוגיקה עסקית לממשק משתמש

- **בדיקות יחידה** — המפתח להצלחה בפיתוח עם סוכני AI
- **יישום מעשי** — בניית מערכת חשבוניות מלאה כדוגמה

1.4 חדשות מעולם ה-AI: סוף שנת 2025

1.4.1 המודלים המובילים

בתחום מודלי השפה, אנו עדים לתחרות עזה בין ענקיות הטכנולוגיה:

- **Gemini 3** — כרגע מוביל בצורה משמעותית בתחום, במיוחד ביכולות עיבוד תמונה וקריאת כתב יד
- **OpenAI O5.2** — גרסה משופרת עם יכולות טובות משמעותית מקודמתה
- **Claude** — מצטיין בהבנת קונטקסט ובכתיבת קוד איכותי

1.4.2 כלים וסביבות חדשות

מספר כלים חדשניים משנים את צורת העבודה שלנו:

טבלה 1: כלים מובילים לעבודה עם סוכני AI

תיאור	שם הכלי
סביבה ייחודית המפעילה מחשב וירטואלי. מאפשרת עבודה עם סוכנים מלאים, מצוינת במיוחד ליצירת מצגות	Manus
פרויקט קוד פתוח שמחקה את Claude Code, מאפשר חיבור לכל ספק מודלים	OpenCode
דפדפן חדש של גוגל המשנה את תפיסת החיפוש באינטרנט עם ממשק גרפי אדפטיבי	Disco
סביבת פיתוח חדשה של גוגל המתחרה ב-VS Code	Anti-Gravity

© Dr. Segal Yoram - כל הזכויות שמורות

1.5 הפרדה בין נתונים לתצוגה: העיקרון המנחה

אחד החידושים המרכזיים שנראה בספר זה הוא ההפרדה המוחלטת בין נתונים לבין הדרך שבה הם מוצגים. כפי שדפדפן Disco של גוגל מדגים, אפשר לקחת את אותם נתונים ולהציג אותם בדרכים שונות לחלוטין בהתאם לצורך.

עיקרון זה הוא הלב של ארכיטקטורת התוכנה שנלמד: הלוגיקה העסקית נשארת קבועה ויציבה, בעוד שממשק המשתמש הוא רק "חלון ראווה" שאפשר להחליף בקלות.

1.6 מבנה הספר

הספר מאורגן בשמונה פרקים, כאשר כל פרק בונה על קודמו:

1. **מבוא** (פרק זה) — הכרת העולם החדש של סוכני AI
2. **ארכיטקטורת שרת-לקוח** — הגדרות בסיסיות ויחסים דינמיים
3. **שלושת הפרימיטיבים** — Tools, Resources, ו-Prompts
4. **ארכיטקטורת תוכנה** — שכבות, מודולריות, וכללי זהב
5. **שכבת התקשורת** — STDIO לעומת HTTP/SSE
6. **SDK וממשק משתמש** — הפרדה נכונה ודרישות ממשק
7. **בדיקות יחידה** — המפתח להצלחה
8. **יישום מעשי** — בניית מערכת חשבוניות

1.7 איך לקרוא את הספר?

מומלץ לקרוא את הפרקים בסדר, שכן כל פרק מניח ידע מהפרקים הקודמים. עם זאת, קוראים מנוסים יכולים לדלג ישירות לפרקים שמעניינים אותם.

בכל פרק תמצאו:

- הסברים תיאורטיים בסגנון נגיש ומעניין
- דוגמאות קוד מעשיות
- תרשימים ודיאגרמות להמחשה
- טבלאות סיכום
- שאלות לחשיבה

בואו נתחיל במסע!

כל הזכויות שמורות - Dr. Segal Yoram ©

2 ארכיטקטורת שרת-לקוח בעולם ה-MCP

2.1 מבוא: למה אנחנו צריכים ארכיטקטורה?

לפני שנצלול לפרטים הטכניים, בואו נשאל שאלה פשוטה: למה בכלל צריך ארכיטקטורה?

דמיינו עיר גדולה ללא תכנון — בתים בנויים באופן אקראי, כבישים שמובילים לשום מקום, ומערכת מים שאף אחד לא מבין כיצד היא פועלת. כך נראה קוד ללא ארכיטקטורה. הוא אולי "עובד", אבל אי אפשר לתחזק אותו, להרחיב אותו, או להבין אותו.

ארכיטקטורת תוכנה היא התכנון העירוני של עולם הקוד — היא קובעת איפה כל רכיב יושב, איך רכיבים מתקשרים זה עם זה, ומה התפקיד של כל אחד מהם.

2.2 ארכיטקטורת שרת-לקוח: העיקרון הבסיסי

2.2.1 הגדרה קלאסית

ארכיטקטורת שרת-לקוח (Client-Server Architecture) היא אחת מארכיטקטורות התוכנה הוותיקות והנפוצות ביותר. העיקרון פשוט:

- **שרת (Server)** — רכיב שמספק שירותים. הוא "מחכה" לבקשות ומגיב להן
- **לקוח (Client)** — רכיב שצורך שירותים. הוא "יוזם" בקשות ומקבל תשובות

זוהי מערכת יחסים אסימטרית: הלקוח תמיד מבקש, השרת תמיד מגיב. כמו מלצר במסעדה — הוא מחכה להזמנה ואז מגיש.

2.2.2 למה זה עובד כל כך טוב?

הסיבה שארכיטקטורה זו שרדה עשרות שנים היא פשטותה. כשאתה יודע מי מבקש ומי מספק, קל לעשות:

1. **הפרדת אחריות** — כל צד יודע מה תפקידו
2. **סקלביליות** — אפשר להוסיף שרתים או לקוחות באופן עצמאי
3. **תחזוקה** — שינוי בצד אחד לא משפיע על השני (כל עוד הממשק נשמר)
4. **אבטחה** — השרת יכול לבדוק הרשאות לפני מתן שירות

2.3 המקרה המיוחד של MCP: תפקידים דינמיים

2.3.1 מעבר לדיכוטומיה הקלאסית

עכשיו מגיע החלק המעניין. ב-MCP (Model Context Protocol), התפקידים אינם קבועים! סוכן AI יכול להיות גם לקוח וגם שרת — תלוי בהקשר.

חשבו על זה כך: רופא במרפאה הוא "ספק שירות" כשהוא מטפל בחולים. אבל אותו רופא הוא "צרכן שירות" כשהוא מזמין בדיקות מעבדה. התפקיד משתנה לפי ההקשר.

טבלה 2: השוואה בין תפקידים קלאסיים לתפקידים ב-MCP

MCP	**ארכיטקטורה קלאסית**	**היבט**
דינמיים: רכיב יכול להחליף תפקידים	קבועים: לקוח תמיד לקוח, שרת תמיד שרת	תפקידים
שני הצדדים יכולים ליזום (דו-כיווני)	רק הלקוח יוזם	יוזמת תקשורת
מורכבת יותר אך גמישה יותר	פשוטה יחסית	מורכבות
סוכני AI, מערכות אוטונומיות	אתרי אינטרנט, מסדי נתונים	שימוש טיפוסי

2.4 שלושת השחקנים ב-MCP

הפרוטוקול מגדיר שלושה תפקידים מרכזיים:

2.4.1 Host — המארח

ה-Host הוא היישום הראשי שמפעיל את כל המערכת. חשבו עליו כמו מנצח בתזמורת — הוא לא מנגן בעצמו, אבל הוא מתאם את כל השאר.

- מנהל את מחזור החיים של כל הרכיבים
- אחראי על אבטחה והרשאות
- מספק את הממשק למשתמש הקצה
- דוגמאות: Claude Desktop, IDE עם תמיכה ב-AI

2.4.2 Client — הלקוח

ה-Client הוא הרכיב שמתחבר לשרתי MCP ומבקש מהם שירותים. ב-MCP, הלקוח הוא בדרך כלל מודל השפה עצמו או הסוכן שמפעיל אותו.

- יוזם חיבורים לשרתים
- שולח בקשות לכלים (Tools), משאבים (Resources), ו-Prompts
- מעבד את התשובות ומחליט על הצעד הבא

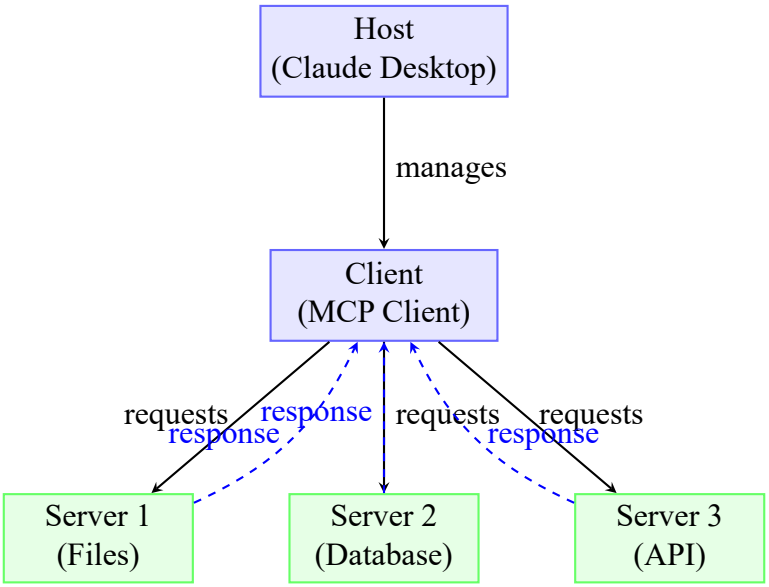
2.4.3 Server — השרת

ה-Server חושף יכולות ושירותים ללקוחות. הוא יכול להיות כל דבר: גישה לקבצים, חיבור למסד נתונים, API חיצוני, או אפילו חיישן פיזי.

- חושף כלים (Tools) — פעולות שאפשר לבצע
- חושף משאבים (Resources) — מידע שאפשר לקרוא
- חושף תבניות (Prompts) — הנחיות מוכנות מראש
- יכול להיות מקומי או מרוחק

2.5 תרשים ארכיטקטורה

התרשים הבא מציג את היחסים בין שלושת השחקנים:



איור 1: ארכיטקטורת MCP: Host מנהל Client שמתקשר עם מספר שרתים. החיצים הרציפים מייצגים בקשות, והמקווקווים מייצגים תשובות.

כל הזכויות שמורות - © Dr. Segal Yoram

2.6 קוד לדוגמה: הגדרת שרת MCP פשוט

הבה נראה כיצד נראה שרת MCP בסיסי בקוד. השרת הבא חושף כלי פשוט לחישוב:

שרת MCP בסיסי: Basic MCP Server

```
# Basic MCP Server Example
# This server exposes a simple calculator tool

from mcp.server import Server, Tool
from mcp.types import TextContent

# Create server instance
server = Server("calculator-server")

# Define a tool that clients can call
@server.tool("add")
async def add_numbers(a: float, b: float) -> TextContent:
    """
␣␣␣␣Add␣two␣numbers␣together.

␣␣␣␣Args:
␣␣␣␣␣␣␣␣a:␣First␣number
␣␣␣␣␣␣␣␣b:␣Second␣number

␣␣␣␣Returns:
␣␣␣␣␣␣␣␣The␣sum␣of␣a␣and␣b
␣␣␣␣"""
    result = a + b
    return TextContent(
        type="text",
        text=f"The␣sum␣of␣{a}␣and␣{b}␣is␣{result}"
    )

# Start the server
if __name__ == "__main__":
    server.run()
```

2.6.1 הסבר הקוד

1. **שורה 7** — יצירת אובייקט השרת עם שם ייחודי
2. **שורות 10-22** — הגדרת כלי (Tool) באמצעות דקורטור. זה מה שהלקוח יוכל לקרוא
3. **שורה 25** — הפעלת השרת שיחכה לחיבורים

2.7 זרימת התקשורת

כיצד בדיוק עוברת בקשה במערכת? הנה הזרימה המלאה:

1. **משתמש שואל שאלה** — "כמה זה 3 + 5?"
2. **Host מעביר ל-Client** — ה-Host (למשל Claude Desktop) מעביר את השאלה ללקוח
3. **Client מזהה צורך בכלי** — הסוכן מבין שצריך לחשב ומחפש כלי מתאים
4. **Client קורא לשרת** — הלקוח שולח בקשה לשרת המחשבון
5. **שרת מבצע ומחזיר** — השרת מחשב 8 = 3 + 5 ומחזיר תוצאה
6. **Client מעבד תשובה** — הסוכן מקבל את התוצאה
7. **Host מציג למשתמש** — "התשובה היא 8"

2.8 יתרונות הארכיטקטורה

2.8.1 מודולריות

כל שרת הוא יחידה עצמאית. אפשר להוסיף, להסיר, או לעדכן שרתים בלי לגעת בשאר המערכת. זה כמו לגו — כל חלק מתחבר בצורה סטנדרטית.

2.8.2 סקלביליות

צריכים יותר יכולות? פשוט הוסיפו עוד שרתים. כל שרת יכול לרוץ על מכונה נפרדת אם צריך.

2.8.3 אבטחה

כל שרת יכול להגדיר הרשאות משלו. ה-Host מנהל את האבטחה הכללית, וכל שרת מגן על המשאבים שלו.

2.8.4 גמישות

אפשר לכתוב שרתים בכל שפת תכנות. כל עוד הם מדברים את פרוטוקול MCP, הם יעבדו יחד.

2.9 אתגרים ופתרונות

2.9.1 אתגר: ניהול מצב

כשיש הרבה שרתים, איך מנהלים את המצב? מי זוכר מה?

פתרון: שימוש ב-Stateless Design — כל בקשה מכילה את כל המידע הנדרש. השרת לא "זוכר" בקשות קודמות.

כל הזכויות שמורות - Dr. Segal Yoram ©

2.9.2 אתגר: תזמון וסנכרון

מה קורה כששני לקוחות רוצים לגשת לאותו משאב?

פתרון: מנגנוני נעילה (Locking) ותורים (Queues). נרחיב על כך בפרק על שכבת התקשורת.

2.10 סיכום הפרק

בפרק זה למדנו:

- ארכיטקטורת שרת-לקוח היא בסיס מוצק לבניית מערכות מבוזרות
- MCP מרחיב את המודל הקלאסי עם תפקידים דינמיים
- שלושת השחקנים — Host, Client, Server — משתפים פעולה בצורה מתואמת
- הארכיטקטורה מאפשרת מודולריות, סקלביליות, ואבטחה

בפרק הבא נצלול לתוך שלושת הפרימיטיבים של MCP: Tools, Resources, ו-Prompts.

3 שלושת הפרימיטיבים של MCP

3.1 מבוא: שפת השיחה בין סוכנים

אם ארכיטקטורת שרת-לקוח היא "הדרך" שבה רכיבים מתקשרים, אז הפרימיטיבים הם "המילים" שבהן הם משתמשים. MCP מגדיר שלושה פרימיטיבים בסיסיים שמהווים את אבני הבניין של כל אינטראקציה:

1. **Tools** — כלים לביצוע פעולות
2. **Resources** — משאבים לקריאת מידע
3. **Prompts** — תבניות להנחיות מוכנות

חשבו על זה כמו על שלושה סוגי משפטים בשפה: פקודות ("עשה!"), שאלות ("מה יש?"), ותבניות מוכנות ("כשמישהו שואל X, ענה Y").

3.2 Tools — כלים: היכולת לפעול

3.2.1 מה זה כלי?

כלי (Tool) הוא פונקציה שהסוכן יכול להפעיל. זה ההבדל בין סוכן "שמדבר" לסוכן "שעושה". כלי מאפשר לסוכן לבצע פעולות בעולם האמיתי: לשלוח אימייל, לשמור קובץ, לבצע חישוב, או לתקשר עם מערכת חיצונית.

3.2.2 מאפייני כלי

כל כלי מוגדר על ידי:

טבלה 3: מאפייני הגדרת כלי ב-MCP

מאפיין	תיאור	דוגמה
שם (name)	מזהה ייחודי לכלי	send_email
תיאור (description)	הסבר מה הכלי עושה	"שולח אימייל לכתובת נתונה"
פרמטרים (parameters)	הקלטים שהכלי מקבל	to, subject, body
סכמה (schema)	הגדרה פורמלית של הפרמטרים	JSON Schema
תוצאה (result)	מה הכלי מחזיר	אישור שליחה או שגיאה

3.2.3 דוגמת קוד: הגדרת כלי

הגדרת כלי לשליחת אימייל: Email Tool Definition

```
from mcp.server import Server
from mcp.types import Tool, TextContent
from pydantic import BaseModel

server = Server("email-server")

# Define input schema using Pydantic
class EmailParams(BaseModel):
    to: str          # Recipient email address
    subject: str     # Email subject line
    body: str        # Email content

@server.tool(
    name="send_email",
    description="Send␣an␣email␣to␣the␣specified␣recipient",
    input_schema=EmailParams.model_json_schema()
)
async def send_email(to: str, subject: str, body: str) -> TextContent:
    """
␣␣␣␣Send␣an␣email␣message.

␣␣␣␣This␣tool␣allows␣the␣AI␣agent␣to␣send␣emails␣on␣behalf
␣␣␣␣of␣the␣user.␣It␣validates␣the␣email␣address␣and␣sends
␣␣␣␣the␣message␣through␣the␣configured␣SMTP␣server.
␣␣␣␣
    # Validate email format
    if "@" not in to:
```

3.2.4 מתי להשתמש בכלים?

כלים מתאימים כאשר:

- צריך לבצע פעולה שמשנה משהו (לא רק לקרוא)
- יש תוצאה ברורה (הצלחה/כישלון)
- הפעולה דורשת פרמטרים ספציפיים
- צריך אינטראקציה עם מערכת חיצונית

3.3 Resources — משאבים: היכולת לקרוא

3.3.1 מה זה משאב?

משאב (Resource) הוא מידע שהסוכן יכול לקרוא. בניגוד לכלי שמבצע פעולה, משאב פשוט מספק נתונים. חשבו על ההבדל בין "תקרא את הקובץ" לבין "תמחק את הקובץ" — הראשון הוא משאב, השני הוא כלי.

3.3.2 סוגי משאבים

טבלה 4: סוגי משאבים נפוצים ב-MCP

דוגמאות	תיאור	סוג
קוד מקור, מסמכים, תמונות	תוכן קבצים ממערכת הקבצים	קבצים
מחירי מניות, מזג אוויר	מידע שמתעדכן כל הזמן	נתוני זמן-אמת
רשומות לקוחות, היסטוריית הזמנות	שאילתות לבסיס נתונים	מסדי נתונים
פרטי משתמש מ-GitHub	נתונים משירותים חיצוניים	ממשקי API
קובצי JSON, משתני סביבה	הגדרות מערכת	תצורה

כל הזכויות שמורות - Dr. Segal Yoram ©

3.3.3 מזהה משאב: URI

כל משאב מזוהה על ידי URI (Uniform Resource Identifier). זה מאפשר לזהות משאבים באופן חד-משמעי:

דוגמאות URI למשאבים: Resource URI Examples

```
# File resources
file:///home/user/documents/report.pdf
file:///C:/Projects/code/main.py

# Database resources
db://myapp/users/123
db://myapp/orders?status=pending

# API resources
api://github/repos/anthropics/claude-code
api://weather/current?city=Tel-Aviv

# Custom protocol resources
myapp://config/settings
myapp://cache/recent-queries
```

3.3.4 דוגמת קוד: חשיפת משאב

שרת משאבים לקבצים: File Resource Server

```
from mcp.server import Server
from mcp.types import Resource, TextContent
from pathlib import Path

server = Server("file-resource-server")

# Define available resources
@server.list_resources()
async def list_available_resources() -> list[Resource]:
    """List␣all␣files␣in␣the␣documents␣folder."""
    docs_path = Path("/home/user/documents")
    resources = []

    for file_path in docs_path.glob("*.txt"):
        resources.append(Resource(
            uri=f"file://{file_path}",
            name=file_path.name,
            description=f"Text␣file:␣{file_path.name}",
            mime_type="text/plain"
        ))

    return resources

# Read a specific resource
@server.read_resource()
async def read_file_resource(uri: str) -> TextContent:
    """Read␣the␣content␣of␣a␣file␣resource."""
    # Extract path from URI
    path = uri.replace("file://", "")

    try:
        content = Path(path).read_text(encoding="utf-8")
        return TextContent(type="text", text=content)
    except FileNotFoundError:
        return TextContent(type="text", text="Error:␣File␣not␣found")
```

3.4 Prompts — תבניות: היכולת להנחות

© Dr. Segal Yoram - כל הזכויות שמורות

3.4.1 מה זה פרומפט?

פרומפט (Prompt) הוא תבנית מוכנה מראש שמספקת הנחיות לסוכן. זה כמו "מתכון" — במקום לתאר כל פעם מחדש איך לבשל פסטה, פשוט אומרים "השתמש במתכון הפסטה".

3.4.2 למה צריך פרומפטים?

1. **עקביות** — אותה משימה תמיד מבוצעת באותו אופן
2. **חיסכון** — לא צריך להקליד הנחיות ארוכות שוב ושוב
3. **מומחיות** — פרומפטים יכולים לכלול ידע מתמחה
4. **אבטחה** — אפשר לוודא שהסוכן פועל בגבולות מוגדרים

3.4.3 מבנה פרומפט

טבלה 5: רכיבי פרומפט ב-MCP

רכיב	תפקיד	דוגמה
שם	מזהה ייחודי	code_review
תיאור	הסבר מה הפרומפט עושה	"סוקר קוד ומציע שיפורים"
ארגומנטים	פרמטרים שאפשר להעביר	שפת תכנות, רמת קפדנות
תוכן	ההנחיות עצמן	טקסט מפורט עם הוראות

3.4.4 דוגמת קוד: הגדרת פרומפט

פרומפט לסקירת קוד: Code Review Prompt

```
from mcp.server import Server
from mcp.types import Prompt, PromptArgument, PromptMessage

server = Server("code-review-server")

@server.list_prompts()
async def list_prompts() -> list[Prompt]:
    """List␣available␣prompts."""
    return [
        Prompt(
            name="code_review",
            description="Review␣code␣for␣bugs␣and␣improvements",
            arguments=[
                PromptArgument(
                    name="language",
                    description="Programming␣language",
                    required=True
                ),
                PromptArgument(
                    name="strictness",
                    description="Review␣strictness:␣low/medium/high",
                    required=False
                )
            ]
        )
    ]

@server.get_prompt()
async def get_prompt(name: str, arguments: dict) -> list[PromptMessage]:
    """Generate␣prompt␣content␣based␣on␣arguments."""
    if name == "code_review":
        language = arguments.get("language", "Python")
        strictness = arguments.get("strictness", "medium")

        return [PromptMessage(
            role="user",
            content=f"""Please␣review␣the␣following␣{language}␣code.

Review␣criteria␣(strictness:␣{strictness}):
1.␣Check␣for␣bugs␣and␣logical␣errors
2.␣Suggest␣performance␣improvements
3.␣Verify␣error␣handling
4.␣Check␣code␣style␣and␣readability
5.␣Identify␣security␣vulnerabilities
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
Provide␣specific␣line␣numbers␣and␣concrete␣suggestions."""
        )]
```

3.5 השוואה בין שלושת הפרימיטיבים

טבלה 6: השוואה מקיפה בין שלושת הפרימיטיבים

Prompts	Resources	Tools	היבט
מתן הנחיות	קריאת מידע	ביצוע פעולה	מטרה
מכוון התנהגות	קורא בלבד	כותב/משנה	כיוון
הנחיות לסוכן	נתונים	שינוי במערכת	תוצאה
תבנית סקירה	קרא קובץ	שלח אימייל	דוגמה
Client/User	Client	Client	יוזם
התחלת משימה	לפי צורך	לפי צורך	תדירות

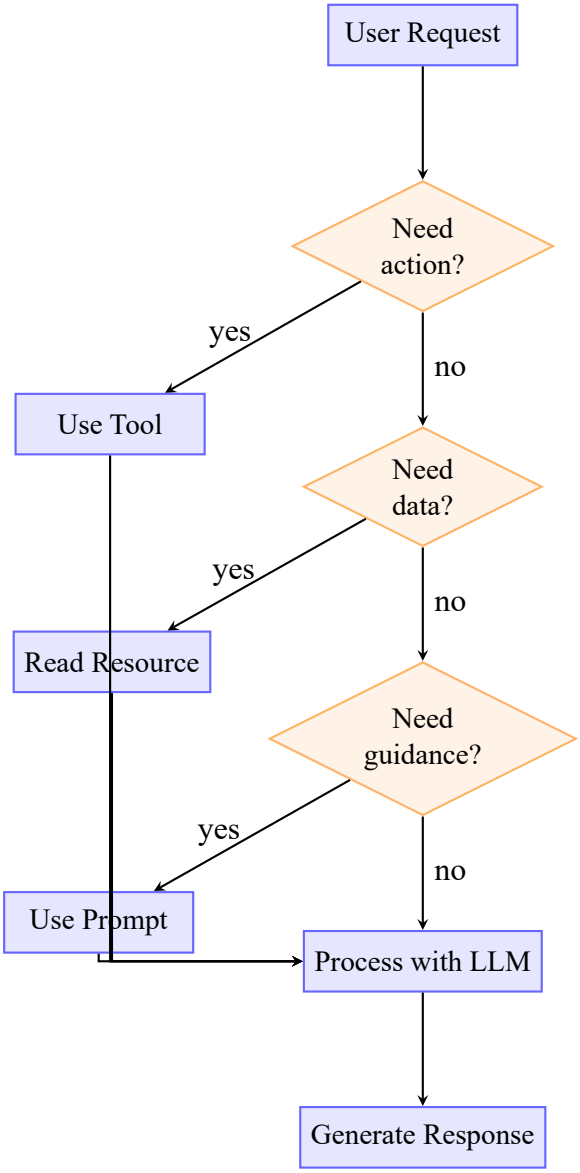
3.6 תרשים זרימה: שימוש בפרימיטיבים

3.7 שילוב הפרימיטיבים בפועל

בעולם האמיתי, סוכן משתמש בשלושת הפרימיטיבים יחד. הנה דוגמה מעשית:

משימה: "כתוב סיכום של כל המיילים שקיבלתי היום ושלח אותו לבוס שלי"

1. **שלב 1 — Prompt:** הסוכן טוען פרומפט "סיכום מיילים" שמגדיר את הפורמט הרצוי
2. **שלב 2 — Resource:** הסוכן קורא את כל המיילים מהיום (משאב: תיבת דואר)
3. **שלב 3 — עיבוד:** הסוכן מעבד את המיילים ומייצר סיכום לפי הפרומפט
4. **שלב 4 — Tool:** הסוכן שולח את הסיכום באימייל לבוס (כלי: שליחת מייל)



איור 2: תרשים זרימה: בחירת פרימיטיב מתאים. הסוכן מחליט איזה פרימיטיב להשתמש בהתאם לצורך — פעולה, מידע, או הנחיה.

© Dr. Segal Yoram - כל הזכויות שמורות

3.8 סיכום הפרק

למדנו על שלושת אבני הבניין של MCP:

- **Tools** — כלים לביצוע פעולות. הם מאפשרים לסוכן "לעשות" ולא רק "לדבר"
- **Resources** — משאבים לקריאת מידע. הם מספקים לסוכן גישה לנתונים
- **Prompts** — תבניות להנחיות. הם מכוונים את התנהגות הסוכן

שלושת הפרימיטיבים יחד יוצרים שפה עשירה שמאפשרת לסוכני AI לתקשר עם העולם החיצון בצורה מובנית ובטוחה.

בפרק הבא נצלול לארכיטקטורת התוכנה עצמה — כיצד לארגן את הקוד בשכבות נפרדות ולשמור על מודולריות.

4 ארכיטקטורת תוכנה לסוכני AI

4.1 מבוא: למה ארכיטקטורה חשובה כל כך?

בפרקים הקודמים למדנו על הפרוטוקול והפרימיטיבים. עכשיו מגיע הזמן לשאול: איך בונים תוכנה אמיתית סביב כל זה?

כשאתם בונים בית, אתם לא מתחילים להניח לבנים באקראי. קודם יש תכנית אדריכלית שמגדירה היכן יהיו הקירות, הצנרת, והחשמל. באותו אופן, ארכיטקטורת תוכנה מגדירה איך הקוד מאורגן, איך רכיבים מתקשרים, ואיך המערכת יכולה לגדול.

4.1.1 העיקרון המנחה: מודולריות

מודולריות היא עקרון תכנון שבו מערכת מחולקת ליחידות עצמאיות (מודולים), כאשר כל מודול אחראי על משימה אחת ומוגדרת היטב.

היתרון המרכזי: אפשר להחליף, לשנות, או לבדוק כל מודול בנפרד מבלי לגעת באחרים.

4.2 ארכיטקטורה שכבתית: Clean Architecture

4.2.1 מהי ארכיטקטורה שכבתית?

ארכיטקטורה שכבתית (Layered Architecture) מחלקת את הקוד לשכבות, כאשר כל שכבה אחראית על סוג מסוים של לוגיקה. הרעיון: שכבות עליונות משתמשות בשכבות תחתונות, אבל לא להפך.

4.2.2 ארבע השכבות

טבלה 7: ארבע שכבות הארכיטקטורה הנקייה

שכבה	אחריות	דוגמאות
UI Layer	ממשק משתמש ותצוגה	CLI, GUI, Web API
Domain Layer	לוגיקה עסקית	חישובים, חוקי עסק, עיבוד
Data Layer	גישה לנתונים	Repository, שמירה וטעינה
Infrastructure	שירותים חיצוניים	API, מסד נתונים, קבצים

4.2.3 תרשים מבנה הפרויקט

הנה מבנה תיקיות מומלץ לפרויקט MCP:

מבנה תיקיות לפרויקט MCP: Project Directory Structure

```
project_root/├──
 src/│├──
    cli/                    # UI Layer - User Interface│├──
       main.py             # Entry point│├──
       handlers/           # Command handlers│└──
       services/           # UI services│└├──

    domain/                 # Domain Layer - Business Logic│├──
       models/             # Domain entities│└──
       services/           # Business services│└├──

    data/                   # Data Layer - Data Access│├──
       repositories/       # Repository implementations│└├──
          interfaces/     # Abstract interfaces│└──
          implementations/│└──
       orm/                # ORM models (if using SQL)│└├──

    infrastructure/         # Infrastructure Layer│├──
       mcp_servers/        # MCP server implementations│├──
       api_clients/        # External API integrations│└──
       database/           # Database connections│└──

    shared/                 # Cross-Cutting Concerns│├──
        config/             # Configuration management│├──
        logging/            # Logging infrastructure│├──
        threading/          # Multi-threading utilities│└──
        utils/              # Utility functions│├──

 config/                     # Configuration files├──
 tests/                      # Test suite├──
 logs/                       # Log files└──
 requirements.txt            # Dependencies
```

4.3 כלל הזהב: מקסימום 150 שורות לקובץ

© Dr. Segal Yoram - כל הזכויות שמורות

4.3.1 למה דווקא 150?

מגבלת 150 שורות לקובץ היא לא מספר שרירותי. היא מבוססת על:

1. **קריאות** — מוח אנושי יכול להחזיק בזיכרון קצר בערך 7 פריטים. קובץ קצר נכנס לראש
2. **אחריות יחידה** — קשה לקובץ קצר "לעשות יותר מדי"
3. **בדיקות** — קל יותר לבדוק יחידות קטנות
4. **תיעוד** — קל יותר לתעד מודול ממוקד

4.3.2 מה עושים כשצריך יותר?

כשקובץ עובר 150 שורות, זה סימן שצריך לפצל:

Before: File Too Large לפני: קובץ גדול מדי:

```
# algorithm.py - 250 lines - TOO LARGE!

class DataProcessor:
    def __init__(self): ...
    def load_data(self): ...
    def clean_data(self): ...
    def validate_data(self): ...
    def process(self): ...
    def save_results(self): ...

# Plus many utility functions...
def calculate_metrics(): ...
def generate_report(): ...
def export_to_csv(): ...
# ... and more
```

After: Split into Focused Files :אחרי: פיצול לקבצים ממוקדים

```
# data_processor/__init__.py
from .loader import DataLoader
from .cleaner import DataCleaner
from .validator import DataValidator
from .processor import DataProcessor
from .exporter import DataExporter

__all__ = [
    'DataLoader', 'DataCleaner', 'DataValidator',
    'DataProcessor', 'DataExporter'
]

# data_processor/loader.py (~40 lines)
# data_processor/cleaner.py (~35 lines)
# data_processor/validator.py (~30 lines)
# data_processor/processor.py (~45 lines)
# data_processor/exporter.py (~40 lines)
```

4.4 דפוס Repository: הפרדת נתונים מלוגיקה

4.4.1 מה זה Repository?

דפוס ה-Repository מפריד בין הלוגיקה העסקית לבין הדרך שבה נתונים נשמרים ונטענים. זה מאפשר להחליף את מקור הנתונים (למשל מ-Excel ל-SQL) בלי לשנות את הלוגיקה.

כל הזכויות שמורות - Dr. Segal Yoram ©

4.4.2 ממשק מופשט

ממשק Repository מופשט: Abstract Repository Interface

```
from abc import ABC, abstractmethod
from typing import List, Optional, TypeVar, Generic

T = TypeVar('T')

class IRepository(ABC, Generic[T]):
    """
␣␣␣␣Abstract␣repository␣interface.

␣␣␣␣This␣interface␣defines␣the␣contract␣for␣data␣access.
␣␣␣␣Any␣implementation␣(SQL,␣Excel,␣JSON)␣must␣follow␣this.
␣␣␣␣"""

    @abstractmethod
    def get_by_id(self, entity_id: int) -> Optional[T]:
        """Get␣entity␣by␣unique␣ID."""
        pass

    @abstractmethod
    def get_all(self, limit: int = None) -> List[T]:
        """Get␣all␣entities,␣optionally␣limited."""
        pass

    @abstractmethod
    def create(self, entity: T) -> T:
        """Create␣a␣new␣entity."""
        pass

    @abstractmethod
    def update(self, entity: T) -> T:
        """Update␣an␣existing␣entity."""
        pass

    @abstractmethod
    def delete(self, entity_id: int) -> bool:
        """Delete␣entity␣by␣ID."""
        pass
```

4.4.3 מימוש קונקרטי

מימוש Repository ל-SQL: SQL Repository Implementation

```
from .interfaces import IRepository
from sqlalchemy.orm import Session
from typing import List, Optional

class SQLInvoiceRepository(IRepository[Invoice]):
    """
␣␣␣␣SQL␣implementation␣of␣Invoice␣repository.

␣␣␣␣Uses␣SQLAlchemy␣for␣database␣operations.
␣␣␣␣"""

    def __init__(self, session: Session):
        self._session = session

    def get_by_id(self, invoice_id: int) -> Optional[Invoice]:
        return self._session.query(Invoice).get(invoice_id)

    def get_all(self, limit: int = None) -> List[Invoice]:
        query = self._session.query(Invoice)
        if limit:
            query = query.limit(limit)
        return query.all()

    def create(self, invoice: Invoice) -> Invoice:
        self._session.add(invoice)
        self._session.commit()
        return invoice

    def update(self, invoice: Invoice) -> Invoice:
        self._session.merge(invoice)
        self._session.commit()
        return invoice

    def delete(self, invoice_id: int) -> bool:
        invoice = self.get_by_id(invoice_id)
        if invoice:
            self._session.delete(invoice)
            self._session.commit()
            return True
        return False
```


© Dr. Segal Yoram - כל הזכויות שמורות

4.5 הזרקת תלויות: Dependency Injection

4.5.1 הבעיה: קוד צמוד

כשקוד יוצר את התלויות שלו בעצמו, הוא "צמוד" אליהן:

קוד צמוד — בעייתי: Tightly Coupled Code - Problematic

```
# BAD: Tightly coupled
class InvoiceService:
    def __init__(self):
        # Creates its own dependencies - hard to test/change!
        self.repository = SQLInvoiceRepository()
        self.email_sender = SMTPEmailSender()
        self.logger = FileLogger()
```

4.5.2 הפתרון: הזרקת תלויות

הזרקת תלויות — נכון: Dependency Injection - Correct

```
# GOOD: Dependencies injected
class InvoiceService:
    """
␣␣␣␣Invoice␣business␣logic␣service.

␣␣␣␣Dependencies␣are␣injected,␣making␣this␣class:
␣␣␣␣-␣Easy␣to␣test␣(inject␣mocks)
␣␣␣␣-␣Flexible␣(swap␣implementations)
␣␣␣␣-␣Decoupled␣from␣infrastructure
␣␣␣␣"""

    def __init__(
        self,
        repository: IRepository[Invoice],
        email_sender: IEmailSender,
        logger: ILogger
    ):
        self._repository = repository
        self._email_sender = email_sender
        self._logger = logger

    def create_invoice(self, data: dict) -> Invoice:
        self._logger.info(f"Creating␣invoice:␣{data}")

        invoice = Invoice(**data)
        saved = self._repository.create(invoice)

        self._email_sender.send(
            to=invoice.customer_email,
            subject=f"Invoice␣#{saved.id}",
            body=f"Your␣invoice␣is␣ready"
        )

        return saved
```


© Dr. Segal Yoram - כל הזכויות שמורות

4.5.3 מיכל DI פשוט

מיכל הזרקת תלויות: Simple DI Container

```
class DIContainer:
    """
␣␣␣␣Lightweight␣dependency␣injection␣container.

␣␣␣␣Supports␣singleton␣and␣transient␣lifetimes.
␣␣␣␣"""

    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._singletons = {}
            cls._instance._factories = {}
        return cls._instance

    def register_singleton(self, interface, implementation):
        """Register␣a␣singleton␣(one␣instance␣shared)."""
        self._factories[interface] = ('singleton', implementation)

    def register_transient(self, interface, implementation):
        """Register␣a␣transient␣(new␣instance␣each␣time)."""
        self._factories[interface] = ('transient', implementation)

    def resolve(self, interface):
        """Resolve␣a␣dependency."""
        scope, impl = self._factories[interface]

        if scope == 'singleton':
            if interface not in self._singletons:
                self._singletons[interface] = impl()
            return self._singletons[interface]
        else:
            return impl()

# Usage
container = DIContainer()
container.register_singleton(IRepository, SQLInvoiceRepository)
container.register_transient(InvoiceService, InvoiceService)
```

4.6 ניהול משאבים בסביבה רב-תהליכית

4.6.1 הבעיה: גישה במקביל

כשיש מספר סוכנים שרצים במקביל, הם עלולים לגשת לאותו משאב בו-זמנית. זה יכול ליצור בעיות:

- **קריאה-כתיבה** — אחד קורא בזמן שאחר כותב
- **כתיבה-כתיבה** — שניים כותבים ודורסים זה את זה
- **Deadlock** — שניים מחכים זה לזה לנצח

כל הזכויות שמורות - Dr. Segal Yoram ©

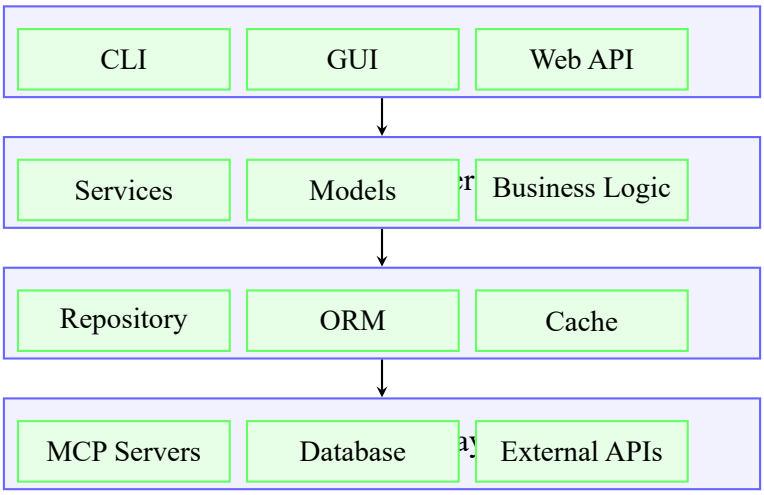
Resource Manager Pattern :מנהל משאבים

```
import threading
from contextlib import contextmanager
from typing import Dict

class ResourceManager:
    """
␣␣␣␣Singleton␣providing␣access␣to␣all␣resource␣locks.

␣␣␣␣Centralizes␣lock␣management␣for␣thread␣safety.
␣␣␣␣"""

    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
                    cls._instance._file_locks: Dict[str, threading.RLock
                        ] = {}
                    cls._instance._manager_lock = threading.Lock()
        return cls._instance

    @contextmanager
    def lock_file(self, file_path: str):
        """
␣␣␣␣␣␣␣␣Lock␣a␣specific␣file␣for␣exclusive␣access.

␣␣␣␣␣␣␣␣Uses␣RLock␣for␣reentrant␣locking␣(same␣thread␣can
␣␣␣␣␣␣␣␣acquire␣multiple␣times).
␣␣␣␣␣␣␣␣"""
        normalized = file_path.lower()

        with self._manager_lock:
            if normalized not in self._file_locks:
                self._file_locks[normalized] = threading.RLock()
            lock = self._file_locks[normalized]

        lock.acquire()
        try:
            yield
        finally:
            lock.release()
```



```
# Usage
manager = ResourceManager()
with manager.lock_file("invoices.xlsx"):
```

4.7 תרשים ארכיטקטורה מלא



איור 3: ארכיטקטורה שכבתית מלאה. כל שכבה תלויה רק בשכבה שמתחתיה, מה שמאפשר החלפה והרחבה קלה.

4.8 סיכום הפרק

בפרק זה למדנו את עקרונות הארכיטקטורה לבניית מערכות סוכני AI:

- **ארכיטקטורה שכבתית** — הפרדה ל-UI, Domain, Data, Infrastructure
- **כלל 150 שורות** — קבצים קטנים וממוקדים
- **דפוס Repository** — הפרדה בין לוגיקה לנתונים
- **הזרקת תלויות** — קוד גמיש וניתן לבדיקה
- **ניהול משאבים** — בטיחות בסביבה רב-תהליכית

בפרק הבא נצלול לשכבת התקשורת — כיצד הרכיבים מדברים זה עם זה.

כל הזכויות שמורות - © Dr. Segal Yoram

5 שכבת התקשורת

5.1 מבוא: איך הודעות עוברות?

דמיינו שני אנשים שרוצים לדבר. הם יכולים לדבר פנים אל פנים, בטלפון, או דרך מכתבים. כל שיטה מתאימה למצב אחר. באותו אופן, רכיבי MCP יכולים לתקשר בשיטות שונות — וזה מה שנלמד בפרק זה.

5.2 שתי שכבות התקשורת

MCP תומך בשתי שכבות תקשורת עיקריות:

טבלה 8: השוואת שכבות תקשורת ב-MCP

HTTP/SSE	STDIO	היבט
רשת — מכונות שונות	מקומי — אותו מחשב	סוג חיבור
תלויה ברשת	מהירה מאוד	מהירות
מורכבת יותר	פשוטה	מורכבות
שירותי ענן, מערכות מבוזרות	פיתוח, כלים מקומיים	שימוש טיפוסי
דורשת הצפנה ואימות	מובנית (אותו מחשב)	אבטחה

5.3 STDIO: תקשורת מקומית

5.3.1 מה זה STDIO?

STDIO הוא קיצור של Standard Input/Output. זו הדרך הכי פשוטה שתוכנות מדברות – אחת כותבת לפלט, השנייה קוראת מהקלט. כמו צינור מים שמחבר שתי ברזים.

5.3.2 איך זה עובד?

1. ה-Host מפעיל את השרת כתהליך נפרד
2. השרת כותב הודעות ל-stdout
3. ה-Host קורא מ-stdout של השרת
4. ה-Host כותב בקשות ל-stdin של השרת
5. השרת קורא מ-stdin

MCP Server with STDIO :STDIO עם MCP שרת

```
import sys
import json

def main():
    """
␣␣␣␣Simple␣MCP␣server␣using␣STDIO␣transport.

␣␣␣␣Reads␣JSON-RPC␣messages␣from␣stdin,
␣␣␣␣writes␣responses␣to␣stdout.
␣␣␣␣"""
    while True:
        # Read line from stdin
        line = sys.stdin.readline()
        if not line:
            break  # EOF - client disconnected

        try:
            # Parse JSON-RPC request
            request = json.loads(line)
            method = request.get("method")
            params = request.get("params", {})

            # Handle request
            if method == "tools/list":
                result = list_tools()
            elif method == "tools/call":
                result = call_tool(params)
            else:
                result = {"error": "Unknown␣method"}

            # Write JSON-RPC response to stdout
            response = {
                "jsonrpc": "2.0",
                "id": request.get("id"),
                "result": result
            }
            print(json.dumps(response), flush=True)

        except json.JSONDecodeError:
            print(json.dumps({"error": "Invalid␣JSON"}), flush=True)

if __name__ == "__main__":
    main()
```


© Dr. Segal Yoram - כל הזכויות שמורות

5.3.3 יתרונות STDIO

- **פשטות** — אין צורך בקונפיגורציה של פורטים או כתובות
- **בידוד** — כל שרת רץ בתהליך נפרד
- **אמינות** — אין בעיות רשת
- **דיבוג** — קל לראות את ההודעות שעוברות

5.4 HTTP/SSE: תקשורת ברשת

5.4.1 מה זה HTTP/SSE?

HTTP אתם כבר מכירים מהאינטרנט. SSE (Server-Sent Events) הוא מנגנון שמאפשר לשרת לשלוח הודעות ללקוח באופן יזום, בלי שהלקוח יבקש.

5.4.2 למה צריך SSE?

HTTP רגיל הוא request-response — הלקוח שואל, השרת עונה. אבל מה אם השרת רוצה לעדכן את הלקוח על משהו חדש? כאן נכנס SSE:

- **בקשות** — הלקוח שולח ב-HTTP POST
- **תשובות ועדכונים** — השרת שולח דרך SSE

HTTP/SSE MCP Server :HTTP/SSE שרת MCP עם

```
from fastapi import FastAPI, Request
from sse_starlette.sse import EventSourceResponse
import asyncio
import json

app = FastAPI()

# Store for pending responses
responses = asyncio.Queue()

@app.post("/message")
async def handle_message(request: Request):
    """
␣␣␣␣Handle␣incoming␣MCP␣messages␣via␣HTTP␣POST.
␣␣␣␣"""
    body = await request.json()
    method = body.get("method")

    if method == "tools/list":
        result = await list_tools()
    elif method == "tools/call":
        result = await call_tool(body.get("params", {}))
    else:
        result = {"error": "Unknown␣method"}

    # Queue response for SSE delivery
    await responses.put({
        "id": body.get("id"),
        "result": result
    })

    return {"status": "accepted"}

@app.get("/events")
async def event_stream():
    """
␣␣␣␣SSE␣endpoint␣for␣server-to-client␣messages.
␣␣␣␣"""
    async def generate():
        while True:
            # Wait for responses or notifications
            response = await responses.get()
            yield {
                "event": "message",
                "data": json.dumps(response)
            }
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
    return EventSourceResponse(generate())
```

5.5 סנכרון ונעילות

5.5.1 הבעיה: גישה במקביל

כשיש מספר לקוחות או תהליכים שניגשים לאותו משאב, עלולות להיווצר בעיות:

Race Condition — מצב שבו תוצאת הפעולה תלויה בסדר הביצוע של תהליכים במקביל, מה שמוביל לתוצאות לא צפויות.

5.5.2 סוגי נעילות

טבלה 9: סוגי נעילות וייעודן

שימוש	התנהגות	סוג נעילה
גישה למשאב יחיד	נעילה בסיסית — אחד בכל פעם	Lock
פונקציות רקורסיביות	נעילה חוזרת — אותו תהליך יכול לנעול שוב	RLock
הגבלת קצב, connection pool	נעילה מוגבלת — עד N במקביל	Semaphore
תיאום בין תהליכים	סיגנל — מחכה להודעה	Event

5.5.3 דוגמת נעילה לקובץ

File Locking for Safe Access :נעילת קובץ לגישה בטוחה

```
import threading
from pathlib import Path
from contextlib import contextmanager

class FileLockManager:
    """
␣␣␣␣Per-file␣locking␣with␣reentrant␣support.

␣␣␣␣Each␣file␣gets␣its␣own␣lock,␣allowing␣concurrent
␣␣␣␣access␣to␣different␣files␣while␣protecting␣same␣file.
␣␣␣␣"""

    def __init__(self):
        self._locks = {}
        self._manager_lock = threading.Lock()

    @contextmanager
    def lock(self, file_path: str):
        """
␣␣␣␣␣␣␣␣Acquire␣lock␣for␣specific␣file.

␣␣␣␣␣␣␣␣Usage:
␣␣␣␣␣␣␣␣␣␣␣␣␣␣with␣lock_manager.lock("data.json"):
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣#␣Safe␣to␣read/write␣data.json␣here
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣process_file()
␣␣␣␣␣␣␣␣"""
        normalized = str(Path(file_path).resolve())

        with self._manager_lock:
            if normalized not in self._locks:
                self._locks[normalized] = threading.RLock()
            lock = self._locks[normalized]

        lock.acquire()
        try:
            yield
        finally:
            lock.release()

# Usage example
lock_manager = FileLockManager()

def save_invoice(invoice_data: dict):
    with lock_manager.lock("invoices.json"):
        # Read existing data
        data = json.loads(Path("invoices.json").read_text())
        # Add new invoice
        data["invoices"].append(invoice_data)
```


© Dr. Segal Yoram - כל הזכויות שמורות

5.6 הגבלת קצב: Rate Limiting

5.6.1 למה צריך הגבלת קצב?

שירותים חיצוניים (כמו API של OpenAI או Google) מגבילים את מספר הבקשות שאפשר לשלוח. אם נשלח יותר מדי — נחסם.

Rate-Limited Queue :תור עם הגבלת קצב

```
import threading
import time
from concurrent.futures import ThreadPoolExecutor, Future
from typing import Callable

class RateLimitedQueue:
    """
␣␣␣␣Queue␣that␣limits␣request␣rate␣to␣external␣services.

␣␣␣␣Ensures␣minimum␣time␣between␣consecutive␣calls.
␣␣␣␣"""

    def __init__(
        self,
        rate_limit_seconds: float = 1.0,
        max_workers: int = 1
    ):
        self._rate_limit = rate_limit_seconds
        self._executor = ThreadPoolExecutor(max_workers=max_workers)
        self._last_call = 0.0
        self._lock = threading.Lock()

    def submit(self, fn: Callable, *args, **kwargs) -> Future:
        """
␣␣␣␣␣␣␣␣Submit␣a␣task␣with␣rate␣limiting.

␣␣␣␣␣␣␣␣Returns␣a␣Future␣that␣can␣be␣awaited␣for␣result.
␣␣␣␣␣␣␣␣"""
        with self._lock:
            now = time.time()
            wait_time = self._rate_limit - (now - self._last_call)

            if wait_time > 0:
                time.sleep(wait_time)

            self._last_call = time.time()

        return self._executor.submit(fn, *args, **kwargs)

# Usage
api_queue = RateLimitedQueue(rate_limit_seconds=0.5)

def call_external_api(prompt: str) -> str:
    # This will be rate-limited automatically
    future = api_queue.submit(api_client.generate, prompt)
    return future.result()
```


© Dr. Segal Yoram - כל הזכויות שמורות

5.7 Watchdog: זיהוי תקיעות

5.7.1 מה זה Watchdog?

Watchdog (כלב שמירה) הוא מנגנון שמזהה כשתהליך "נתקע" ולא מגיב. הוא עובד על עקרון הפעימה (Heartbeat) — כל תהליך צריך לדווח שהוא בחיים.

Watchdog Mechanism :Watchdog מנגנון

```
import threading
import time
from dataclasses import dataclass
from typing import Dict, Optional

@dataclass
class ThreadInfo:
    name: str
    started_at: float
    last_heartbeat: float
    checkpoint: str = ""

class Watchdog:
    """
␣␣␣␣Monitors␣worker␣threads␣and␣detects␣deadlocks.

␣␣␣␣Workers␣must␣call␣heartbeat()␣periodically,
␣␣␣␣otherwise␣watchdog␣considers␣them␣stuck.
␣␣␣␣"""

    def __init__(self, timeout_seconds: float = 120.0):
        self.timeout = timeout_seconds
        self._threads: Dict[int, ThreadInfo] = {}
        self._lock = threading.Lock()
        self._running = False

    def start(self):
        self._running = True
        monitor = threading.Thread(target=self._monitor_loop)
        monitor.daemon = True
        monitor.start()

    def heartbeat(self, thread_id: int, checkpoint: str = ""):
        """
␣␣␣␣␣␣␣␣Worker␣calls␣this␣to␣report␣it's␣alive.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣thread_id:␣Unique␣thread␣identifier
␣␣␣␣␣␣␣␣␣␣␣␣checkpoint:␣What␣the␣thread␣is␣currently␣doing
␣␣␣␣␣␣␣␣"""
        with self._lock:
            if thread_id in self._threads:
                self._threads[thread_id].last_heartbeat = time.time()
                self._threads[thread_id].checkpoint = checkpoint

    def _monitor_loop(self):
        while self._running:
            self._check_threads()
            time.sleep(5.0)
```


© Dr. Segal Yoram - כל הזכויות שמורות

5.8 סיכום הפרק

בפרק זה למדנו על שכבת התקשורת:

- **STDIO** — תקשורת מקומית פשוטה ומהירה
- **HTTP/SSE** — תקשורת רשת לשירותים מבוזרים
- **נעילות** — הגנה על משאבים משותפים
- **הגבלת קצב** — מניעת עומס על שירותים חיצוניים
- **Watchdog** — זיהוי תהליכים תקועים

בפרק הבא נלמד על SDK וממשק משתמש — איך להפריד בין הלוגיקה לתצוגה.

6 SDK וממשק משתמש

6.1 מבוא: למה להפריד SDK מ-GUI?

נניח שבניתם אפליקציה נהדרת לניהול חשבוניות. היא עובדת מצוין עם ממשק דסקטופ. עכשיו הבוס מבקש גרסת ווב. ואחר כך גרסה לטרמינל. ואז API לאינטגרציה עם מערכות אחרות.

אם הקוד שלכם מעורבב — לוגיקה עסקית ביחד עם כפתורים ותצוגה — תצטרכו לשכתב הכל מחדש לכל ממשק. זה בזבוז זמן עצום.

הפתרון: SDK (Software Development Kit) — שכבה שמכילה את כל הלוגיקה, ש-**כל** ממשק יכול להשתמש בה.

6.2 מה זה SDK?

SDK הוא אוסף של פונקציות, קלאסים, וממשקים שמספקים גישה ליכולות המערכת בצורה מאורגנת וקלה לשימוש.

6.2.1 מאפייני SDK טוב

6.3 ארכיטקטורת SDK

טבלה 10: מאפייני SDK איכותי

דוגמה	משמעות	מאפיין
אותו קוד עובד בטרמינל ובווב	לא תלוי ב-GUI ספציפי	עצמאי מממשק
create_invoice(), לא do_stuff()	פונקציות עם שמות ברורים	ממשק ברור
docstrings, דוגמאות	כל פונקציה מתועדת	מתועד
אין תלות בממשק גרפי	קל לכתוב טסטים	ניתן לבדיקה
שינויים לא שוברים קוד קיים	שומר על תאימות לאחור	גרסאות

6.3.1 מבנה מומלץ

מבנה SDK: SDK Structure

```
sdk/├──
 __init__.py          # Public API exports├──
 client.py            # Main SDK client class├──
 models/              # Data models (Pydantic)│├──
    __init__.py│├──
    invoice.py│├──
    customer.py│└──
    product.py├──
 services/            # Business logic│├──
    __init__.py│├──
    invoice_service.py│├──
    customer_service.py│└──
    report_service.py├──
 exceptions.py        # Custom exceptions└──
 utils.py             # Helper functions
```

6.3.2 קלאס SDK ראשי

קלאס SDK ראשי: Main SDK Client Class

```
from typing import List, Optional
from .models import Invoice, Customer, Product
from .services import InvoiceService, CustomerService
from .exceptions import SDKError, NotFoundError

class InvoiceSDK:
    """
␣␣␣␣Main␣SDK␣client␣for␣invoice␣management.

␣␣␣␣This␣class␣provides␣all␣business␣operations.
␣␣␣␣GUI␣applications␣use␣this␣class,␣never␣accessing
␣␣␣␣database␣or␣files␣directly.

␣␣␣␣Example:
␣␣␣␣␣␣␣␣sdk␣=␣InvoiceSDK()
␣␣␣␣␣␣␣␣invoice␣=␣sdk.create_invoice(
␣␣␣␣␣␣␣␣␣␣␣␣␣␣customer_id=123,
␣␣␣␣␣␣␣␣␣␣␣␣␣␣items=[{"product_id":␣1,␣"quantity":␣2}]
␣␣␣␣␣␣␣␣)
␣␣␣␣␣␣␣␣print(f"Created invoice #{invoice.id}")
    """

␣␣␣␣def␣__init__(self,␣config_path:␣str␣=␣None):
␣␣␣␣␣␣␣␣self._invoice_service␣=␣InvoiceService()
␣␣␣␣␣␣␣␣self._customer_service␣=␣CustomerService()

␣␣␣␣#␣===␣Invoice␣Operations␣===

␣␣␣␣def␣create_invoice(
␣␣␣␣␣␣␣␣self,
␣␣␣␣␣␣␣␣customer_id:␣int,
␣␣␣␣␣␣␣␣items:␣List[dict]
␣␣␣␣)␣->␣Invoice:
␣␣␣␣␣␣␣␣"""
        Create a new invoice.

        Args:
            customer_id: Customer to bill
            items: List of {"product_id": int, "quantity": int}

        Returns:
            The created Invoice object

        Raises:
            NotFoundError: If customer doesn't␣exist
␣␣␣␣␣␣␣␣␣␣␣␣␣␣SDKError:␣If␣creation␣fails
␣␣␣␣␣␣␣␣"""
␣␣␣␣␣␣␣␣return␣self._invoice_service.create(customer_id,␣items)
```


© Dr. Segal Yoram - כל הזכויות שמורות

6.4 מודלים: Pydantic

6.4.1 למה Pydantic?

Pydantic מאפשר להגדיר מודלים עם:

- אימות טיפוסים אוטומטי
- המרה מ/ל-JSON
- ערכי ברירת מחדל
- תיעוד מובנה

Invoice Model with Pydantic :Pydantic מודל חשבונית עם

```
from pydantic import BaseModel, Field
from datetime import datetime
from typing import List, Optional
from enum import Enum

class InvoiceStatus(str, Enum):
    DRAFT = "draft"
    SENT = "sent"
    PAID = "paid"
    CANCELLED = "cancelled"

class InvoiceItem(BaseModel):
    """Single␣line␣item␣in␣an␣invoice."""
    product_id: int
    product_name: str
    quantity: int = Field(gt=0, description="Must␣be␣positive")
    unit_price: float = Field(gt=0)
    total: float = Field(default=0)

    def model_post_init(self, __context):
        self.total = self.quantity * self.unit_price

class Invoice(BaseModel):
    """
␣␣␣␣Invoice␣model␣with␣validation.

␣␣␣␣All␣fields␣are␣validated␣automatically␣when
␣␣␣␣creating␣an␣instance.
␣␣␣␣"""
    id: Optional[int] = None
    customer_id: int
    customer_name: str
    items: List[InvoiceItem]
    subtotal: float = 0
    tax_rate: float = Field(default=0.17, ge=0, le=1)
    tax_amount: float = 0
    total: float = 0
    status: InvoiceStatus = InvoiceStatus.DRAFT
    created_at: datetime = Field(default_factory=datetime.now)
    notes: Optional[str] = None

    def model_post_init(self, __context):
        """Calculate␣totals␣after␣creation."""
        self.subtotal = sum(item.total for item in self.items)
        self.tax_amount = self.subtotal * self.tax_rate
        self.total = self.subtotal + self.tax_amount
```

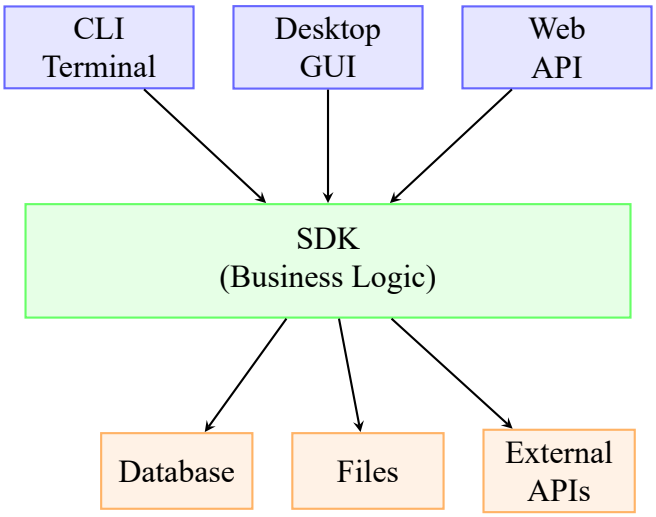

© Dr. Segal Yoram - כל הזכויות שמורות

```
# Usage
invoice = Invoice(
    customer_id=1,
```

6.5 ממשקי משתמש מרובים

6.5.1 העיקרון: SDK אחד, ממשקים רבים

עכשיו שיש לנו SDK, אפשר לבנות ממשקים שונים שכולם משתמשים באותו קוד:



איור 4: ארכיטקטורת SDK: ממשקים שונים משתמשים באותה לוגיקה. שינוי בלוגיקה משפיע על כל הממשקים אוטומטית.

6.5.2 ממשק CLI

ממשק שורת פקודה: Command Line Interface

```
import argparse
from sdk import InvoiceSDK

def main():
    """CLI␣interface␣using␣the␣SDK."""
    parser = argparse.ArgumentParser(description="Invoice␣Manager")
    subparsers = parser.add_subparsers(dest="command")

    # Create invoice command
    create_parser = subparsers.add_parser("create", help="Create␣invoice
        ")
    create_parser.add_argument("--customer", type=int, required=True)
    create_parser.add_argument("--items", type=str, required=True)

    # List invoices command
    list_parser = subparsers.add_parser("list", help="List␣invoices")
    list_parser.add_argument("--status", type=str)

    args = parser.parse_args()

    # Initialize SDK
    sdk = InvoiceSDK()

    if args.command == "create":
        items = parse_items(args.items)
        invoice = sdk.create_invoice(args.customer, items)
        print(f"Created␣invoice␣#{invoice.id}")
        print(f"Total:␣{invoice.total:.2f}")

    elif args.command == "list":
        invoices = sdk.list_invoices(status=args.status)
        for inv in invoices:
            print(f"#{inv.id}:␣{inv.customer_name}␣-␣{inv.total:.2f}")

if __name__ == "__main__":
    main()
```


© Dr. Segal Yoram - כל הזכויות שמורות

6.5.3 ממשק ווב API

ממשק ווב API: Web API Interface

```
from fastapi import FastAPI, HTTPException
from sdk import InvoiceSDK
from sdk.models import Invoice
from sdk.exceptions import NotFoundError

app = FastAPI(title="Invoice␣API")
sdk = InvoiceSDK()

@app.post("/invoices", response_model=Invoice)
async def create_invoice(customer_id: int, items: list):
    """Create␣a␣new␣invoice␣via␣API."""
    try:
        invoice = sdk.create_invoice(customer_id, items)
        return invoice
    except NotFoundError as e:
        raise HTTPException(status_code=404, detail=str(e))

@app.get("/invoices/{invoice_id}", response_model=Invoice)
async def get_invoice(invoice_id: int):
    """Get␣invoice␣by␣ID."""
    invoice = sdk.get_invoice(invoice_id)
    if not invoice:
        raise HTTPException(status_code=404, detail="Invoice␣not␣found")
    return invoice

@app.get("/invoices", response_model=list[Invoice])
async def list_invoices(status: str = None, limit: int = 100):
    """List␣all␣invoices␣with␣optional␣filters."""
    return sdk.list_invoices(status=status, limit=limit)

@app.get("/reports/monthly")
async def monthly_report(year: int, month: int):
    """Generate␣monthly␣sales␣report."""
    return sdk.generate_monthly_report(year, month)
```

6.6 דרישות ממשק משתמש

6.6.1 רענון נתונים

כשהנתונים משתנים, הממשק צריך להתעדכן:

כל הזכויות שמורות - Dr. Segal Yoram ©

Refresh Mechanism :מנגנון רענון

```
import threading
import time

class GUIRefreshManager:
    """
␣␣␣␣Manages␣automatic␣data␣refresh␣for␣GUI.

␣␣␣␣Polls␣SDK␣for␣changes␣and␣notifies␣UI␣to␣update.
␣␣␣␣"""

    def __init__(self, sdk: InvoiceSDK, interval_seconds: float = 5.0):
        self._sdk = sdk
        self._interval = interval_seconds
        self._callbacks = []
        self._running = False
        self._last_data_hash = None

    def register_callback(self, callback):
        """Register␣a␣function␣to␣call␣when␣data␣changes."""
        self._callbacks.append(callback)

    def start(self):
        """Start␣the␣refresh␣loop."""
        self._running = True
        thread = threading.Thread(target=self._refresh_loop)
        thread.daemon = True
        thread.start()

    def stop(self):
        self._running = False

    def _refresh_loop(self):
        while self._running:
            try:
                # Check for data changes
                current_data = self._sdk.list_invoices()
                current_hash = hash(str(current_data))

                if current_hash != self._last_data_hash:
                    self._last_data_hash = current_hash
                    # Notify all registered callbacks
                    for callback in self._callbacks:
                        callback(current_data)

            except Exception as e:
                print(f"Refresh␣error:␣{e}")
```



```
            time.sleep(self._interval)
```

6.6.2 ייצוא נתונים

ממשק טוב מאפשר לייצא נתונים לפורמטים שונים:

כל הזכויות שמורות - Dr. Segal Yoram ©

Data Export :ייצוא נתונים

```
from pathlib import Path
import json
import csv

class DataExporter:
    """Export␣SDK␣data␣to␣various␣formats."""

    def __init__(self, sdk: InvoiceSDK):
        self._sdk = sdk

    def export_to_json(self, output_path: str):
        """Export␣all␣invoices␣to␣JSON␣file."""
        invoices = self._sdk.list_invoices()
        data = [inv.model_dump() for inv in invoices]

        Path(output_path).write_text(
            json.dumps(data, indent=2, default=str)
        )

    def export_to_csv(self, output_path: str):
        """Export␣invoices␣summary␣to␣CSV."""
        invoices = self._sdk.list_invoices()

        with open(output_path, 'w', newline='', encoding='utf-8') as f:
            writer = csv.writer(f)
            writer.writerow(['ID', 'Customer', 'Total', 'Status', 'Date'
               ])

            for inv in invoices:
                writer.writerow([
                    inv.id,
                    inv.customer_name,
                    inv.total,
                    inv.status.value,
                    inv.created_at.isoformat()
                ])

# Usage
exporter = DataExporter(sdk)
exporter.export_to_json("backup/invoices.json")
exporter.export_to_csv("reports/invoices.csv")
```

6.7 סיכום הפרק

בפרק זה למדנו:

- **SDK** הוא שכבה שמכילה את כל הלוגיקה העסקית
- **הפרדה** מאפשרת לבנות ממשקים שונים בקלות
- **Pydantic** מספק מודלים עם אימות אוטומטי
- **ממשקים מרובים** — CLI, GUI, Web API — משתמשים באותו SDK
- **רענון וייצוא** הם יכולות חיוניות בכל ממשק

בפרק הבא נלמד על בדיקות יחידה — המפתח לקוד אמין ובר-תחזוקה.

כל הזכויות שמורות - Dr. Segal Yoram ©

7 בדיקות יחידה והבטחת איכות

7.1 מבוא: למה בדיקות כל כך חשובות?

"אבל הקוד עובד!" — זו הטענה הנפוצה ביותר נגד כתיבת בדיקות. והתשובה היא: הקוד עובד **עכשיו**. מה יקרה כשתשנו משהו? כשתוסיפו פיצ'ר חדש? כשמישהו אחר יעבוד על הקוד?

בדיקות אוטומטיות הן כמו רשת ביטחון לאקרובטים — הן לא מונעות נפילה, אבל הן מונעות קטסטרופה.

7.1.1 העיקרון המנחה של ד"ר סגל

> "כל פעם שמופיע באג, תייצרו Unit Test שבודק אותו. כך לא תיפלו לאותו בור פעמיים."

זהו עיקרון זהב: במקום רק לתקן באג, הפכו אותו להזדמנות לחזק את המערכת.

7.2 סוגי בדיקות

7.3 Unit Tests: היסוד

7.3.1 מה זה Unit Test?

בדיקת יחידה (Unit Test) בודקת יחידת קוד קטנה — בדרך כלל פונקציה או מתודה אחת — בבידוד מלא משאר המערכת.

7.3.2 מאפייני בדיקה טובה

- **מהירה** — רצה בפחות משנייה
- **מבודדת** — לא תלויה בבדיקות אחרות או בסביבה
- **חוזרתית** — תמיד נותנת אותה תוצאה
- **ברורה** — כשנכשלת, ברור למה
- **ממוקדת** — בודקת דבר אחד

טבלה 11: פירמידת הבדיקות

כמות	מה בודק	סוג
הרבה (בסיס הפירמידה)	פונקציה או קלאס בודד	Unit Tests
בינוני	שילוב בין רכיבים	Integration Tests
מעט (ראש הפירמידה)	זרימה מלאה מקצה לקצה	E2E Tests
לפי צורך	מהירות וצריכת משאבים	Performance Tests

7.3.3 מבנה בדיקה: AAA

מבנה AAA: Arrange-Act-Assert Pattern

```
import pytest
from sdk.models import Invoice, InvoiceItem

def test_invoice_calculates_total_correctly():
    """
␣␣␣␣Test␣that␣invoice␣total␣includes␣items␣and␣tax.

␣␣␣␣AAA␣Pattern:
␣␣␣␣-␣Arrange:␣Set␣up␣the␣test␣data
␣␣␣␣-␣Act:␣Execute␣the␣code␣being␣tested
␣␣␣␣-␣Assert:␣Verify␣the␣results
␣␣␣␣"""
    # Arrange - Set up test data
    items = [
        InvoiceItem(
            product_id=1,
            product_name="Widget",
            quantity=2,
            unit_price=100.0
        ),
        InvoiceItem(
            product_id=2,
            product_name="Gadget",
            quantity=1,
            unit_price=50.0
        )
    ]
```



```
    # Act - Create the invoice
    invoice = Invoice(
```

7.4 מוקים: בידוד תלויות

7.4.1 מה זה Mock?

Mock הוא אובייקט מזויף שמחליף תלות אמיתית. זה מאפשר לבדוק קוד בבידוד, בלי לגשת למסד נתונים, לשלוח אימיילים, או לקרוא ל-API חיצוני.

כל הזכויות שמורות - Dr. Segal Yoram ©

Using Mocks for Isolation :Mock-ב שימוש

```
from unittest.mock import Mock, patch
from sdk import InvoiceSDK
from sdk.models import Invoice, Customer

def test_create_invoice_calls_email_service():
    """
␣␣␣␣Test␣that␣creating␣invoice␣sends␣email␣notification.

␣␣␣␣Uses␣mock␣to␣avoid␣actually␣sending␣email.
␣␣␣␣"""
    # Arrange - Create mocks
    mock_repository = Mock()
    mock_email = Mock()

    # Configure mock behavior
    mock_repository.create.return_value = Invoice(
        id=1,
        customer_id=1,
        customer_name="Test",
        items=[],
        total=100
    )

    # Create SDK with mocked dependencies
    sdk = InvoiceSDK()
    sdk._invoice_service._repository = mock_repository
    sdk._invoice_service._email_sender = mock_email

    # Act - Create invoice
    invoice = sdk.create_invoice(
        customer_id=1,
        items=[{"product_id": 1, "quantity": 1}]
    )

    # Assert - Verify email was sent
    mock_email.send.assert_called_once()
    call_args = mock_email.send.call_args
    assert "Invoice␣#1" in call_args.kwargs.get("subject", "")

def test_get_invoice_returns_none_for_missing():
    """Test␣that␣missing␣invoice␣returns␣None,␣not␣error."""
    # Arrange
    mock_repository = Mock()
    mock_repository.get_by_id.return_value = None

    sdk = InvoiceSDK()
    sdk._invoice_service._repository = mock_repository

    # Act
    result = sdk.get_invoice(999)
```

7.5 בדיקות פרמטריות

7.5.1 למה פרמטרים?

לפעמים צריך לבדוק את אותה לוגיקה עם קלטים שונים. במקום לכתוב עשר בדיקות כמעט זהות, משתמשים בבדיקות פרמטריות:

בדיקות פרמטריות: Parametrized Tests

```
import pytest
from sdk.services import InvoiceService

class TestInvoiceValidation:
    """Tests␣for␣invoice␣validation␣logic."""

    @pytest.mark.parametrize("quantity,expected_valid", [
        (1, True),        # Valid: positive
        (10, True),       # Valid: larger positive
        (0, False),       # Invalid: zero
        (-1, False),      # Invalid: negative
        (-100, False),    # Invalid: large negative
    ])
    def test_quantity_validation(self, quantity, expected_valid):
        """
␣␣␣␣␣␣␣␣Test␣that␣only␣positive␣quantities␣are␣valid.

␣␣␣␣␣␣␣␣This␣single␣test␣function␣runs␣5␣times␣with␣different␣inputs.
␣␣␣␣␣␣␣␣"""
        service = InvoiceService()
        result = service.validate_quantity(quantity)
        assert result == expected_valid

    @pytest.mark.parametrize("tax_rate,expected", [
        (0.0, True),      # Valid: no tax
        (0.17, True),     # Valid: standard VAT
        (0.25, True),     # Valid: high rate
        (-0.1, False),    # Invalid: negative
        (1.5, False),     # Invalid: over 100%
    ])
    def test_tax_rate_validation(self, tax_rate, expected):
        """Test␣valid␣tax␣rate␣ranges."""
        service = InvoiceService()
        result = service.validate_tax_rate(tax_rate)
        assert result == expected
```


כל הזכויות שמורות - Dr. Segal Yoram ©

7.6 בדיקות אינטגרציה

7.6.1 מה בודקות?

בדיקות אינטגרציה בודקות שרכיבים שונים עובדים יחד נכון:

בדיקת אינטגרציה: Integration Test

```
import pytest
from sdk import InvoiceSDK
from sdk.models import Customer

@pytest.fixture
def sdk_with_test_db():
    """
␣␣␣␣Fixture␣that␣provides␣SDK␣connected␣to␣test␣database.

␣␣␣␣The␣database␣is␣reset␣before␣each␣test.
␣␣␣␣"""
    # Use in-memory SQLite for tests
    sdk = InvoiceSDK(db_url="sqlite:///:memory:")
    sdk._setup_test_data()
    return sdk

class TestInvoiceWorkflow:
    """Integration␣tests␣for␣complete␣invoice␣workflows."""

    def test_create_and_retrieve_invoice(self, sdk_with_test_db):
        """
␣␣␣␣␣␣␣␣Test␣complete␣flow:␣create␣->␣retrieve␣->␣verify.
␣␣␣␣␣␣␣␣"""
        sdk = sdk_with_test_db

        # Create customer first
        customer = sdk.create_customer(
            name="Integration␣Test␣Co",
            email="test@example.com"
        )

        # Create invoice for customer
        invoice = sdk.create_invoice(
            customer_id=customer.id,
            items=[
                {"product_id": 1, "quantity": 3, "unit_price": 50.0}
            ]
        )

        # Retrieve and verify
        retrieved = sdk.get_invoice(invoice.id)

        assert retrieved is not None
        assert retrieved.customer_id == customer.id
        assert retrieved.total == 175.5  # 150 + 17% VAT
        assert len(retrieved.items) == 1
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
    def test_invoice_affects_customer_balance(self, sdk_with_test_db):
        """
␣␣␣␣␣␣␣␣Test␣that␣creating␣invoice␣updates␣customer␣balance.
```

7.7 ארגון תיקיית הבדיקות

מבנה תיקיית בדיקות: Test Directory Structure

```
tests/├──
 __init__.py├──
 conftest.py              # Shared fixtures├──
 unit/                    # Unit tests│├──
    __init__.py│├──
    test_models.py│├──
    test_invoice_service.py│└──
    test_validators.py├──
 integration/             # Integration tests│├──
    __init__.py│├──
    test_sdk_workflow.py│└──
    test_database.py├──
 e2e/                     # End-to-end tests│├──
    __init__.py│└──
    test_api_endpoints.py├──
 fixtures/                # Test data│├──
    sample_invoices.json│└──
    sample_customers.json└──
 performance/             # Performance benchmarks├──
     __init__.py└──
     test_benchmarks.py
```

7.8 כיסוי קוד

7.8.1 מהו כיסוי קוד?

כיסוי קוד (Code Coverage) מודד כמה מהקוד נבדק על ידי הבדיקות:

הרצת בדיקות עם כיסוי: Running Tests with Coverage

```
# Install pytest-cov
pip install pytest-cov

# Run tests with coverage report
pytest --cov=src --cov-report=html

# Output example:
# Name                      Stmts   Miss  Cover
# -------------------------------------------
# src/__init__.py               5      0   100%
# src/models/invoice.py        45      3    93%
# src/services/invoice.py      78     12    85%
# src/utils.py                 23      8    65%
# -------------------------------------------
# TOTAL                       151     23    85%
```

7.8.2 יעד כיסוי

- **+80%** — יעד מינימלי לפרויקטים מקצועיים
- **+90%** — מומלץ לקוד קריטי
- **100%** — לא תמיד אפשרי או הכרחי

7.9 פיתוח מונחה בדיקות: TDD

כל הזכויות שמורות - Dr. Segal Yoram ©

7.9.1 מהו TDD?

Test-Driven Development הוא תהליך שבו כותבים את הבדיקה **לפני** הקוד:

1. **אדום** — כתוב בדיקה שנכשלת (הקוד עוד לא קיים)
2. **ירוק** — כתוב קוד מינימלי שעובר את הבדיקה
3. **שיפור** — שפר את הקוד תוך שמירה על הבדיקות

TDD Example :TDD דוגמת

```
# Step 1: RED - Write failing test first
def test_calculate_discount_for_loyal_customer():
    """Loyal␣customers␣(3+␣years)␣get␣10%␣discount."""
    calculator = DiscountCalculator()

    discount = calculator.calculate(
        customer_years=5,
        purchase_amount=1000
    )

    assert discount == 100.0  # 10% of 1000

# Test fails: DiscountCalculator doesn't exist yet!

# Step 2: GREEN - Write minimal code to pass
class DiscountCalculator:
    def calculate(self, customer_years: int, purchase_amount: float) ->
        float:
        if customer_years >= 3:
            return purchase_amount * 0.10
        return 0.0

# Test passes!

# Step 3: REFACTOR - Improve the code
class DiscountCalculator:
    LOYALTY_THRESHOLD_YEARS = 3
    LOYALTY_DISCOUNT_RATE = 0.10

    def calculate(self, customer_years: int, purchase_amount: float) ->
        float:
        """
␣␣␣␣␣␣␣␣Calculate␣discount␣based␣on␣customer␣loyalty.

␣␣␣␣␣␣␣␣Loyal␣customers␣(3+␣years)␣receive␣10%␣discount.
␣␣␣␣␣␣␣␣"""
        if self._is_loyal_customer(customer_years):
            return self._apply_loyalty_discount(purchase_amount)
        return 0.0

    def _is_loyal_customer(self, years: int) -> bool:
        return years >= self.LOYALTY_THRESHOLD_YEARS

    def _apply_loyalty_discount(self, amount: float) -> float:
        return amount * self.LOYALTY_DISCOUNT_RATE

# Test still passes, but code is cleaner!
```


© Dr. Segal Yoram - כל הזכויות שמורות

7.10 סיכום הפרק

בפרק זה למדנו:

- **בדיקות יחידה** הן היסוד לקוד אמין
- **מוקים** מאפשרים בידוד ובדיקה ללא תלויות חיצוניות
- **בדיקות פרמטריות** חוסכות שכפול קוד
- **בדיקות אינטגרציה** בודקות שרכיבים עובדים יחד
- **כיסוי קוד** מודד את איכות הבדיקות
- **TDD** הוא תהליך שמוביל לקוד איכותי יותר

בפרק הבא והאחרון נראה יישום מעשי של כל מה שלמדנו — בניית מערכת חשבוניות מלאה.

8 יישום מעשי: מערכת חשבוניות

8.1 מבוא: מתיאוריה למעשה

בפרקים הקודמים למדנו את כל אבני הבניין: ארכיטקטורת שרת-לקוח, פרימיטיבים, שכבות, SDK, ובדיקות. עכשיו הגיע הזמן לשלב הכל יחד לפרויקט אמיתי — מערכת ניהול חשבוניות מבוססת MCP.

8.1.1 חמשת השלבים של הפרויקט

הפרויקט נבנה בחמישה שלבים מצטברים:

טבלה 12: חמשת שלבי הפרויקט

מה נוסף	מטרה	שלב
שרת MCP ולקוח שמתקשרים	העברת מידע בסיסית	1
יצירה, עדכון, מחיקה של חשבוניות	הוספת כלים	2
קריאת חשבוניות ולקוחות	הוספת משאבים	3
CLI ו-GUI שמשתמשים באותו SDK	SDK וממשקים	4
מערכת בדיקות מלאה	בדיקות ואינטגרציה	5

8.2 שלב 1: תשתית בסיסית

8.2.1 מבנה הפרויקט

מבנה תיקיות שלב 1: Stage 1 Directory Structure

```
invoice-system/├──
 stage1-basic/│├──
    server/│└├──
       __init__.py│└──
       mcp_server.py│├──
    client/│└├──
       __init__.py│└──
       mcp_client.py│└──
    main.py├──
 requirements.txt└──
 README.md
```

8.2.2 שרת MCP בסיסי

שרת בסיסי — שלב 1: Basic MCP Server

```
# server/mcp_server.py
from mcp.server import Server
from mcp.types import TextContent

# Create server instance
server = Server("invoice-server-v1")

@server.tool("ping")
async def ping() -> TextContent:
    """
␣␣␣␣Simple␣ping␣tool␣to␣test␣connectivity.

␣␣␣␣Returns␣a␣pong␣message␣to␣confirm␣server␣is␣alive.
␣␣␣␣"""
    return TextContent(type="text", text="pong␣-␣server␣is␣alive!")

@server.tool("echo")
async def echo(message: str) -> TextContent:
    """
␣␣␣␣Echo␣back␣the␣received␣message.

␣␣␣␣Args:
␣␣␣␣␣␣␣␣message:␣Text␣to␣echo␣back

␣␣␣␣Returns:
␣␣␣␣␣␣␣␣The␣same␣message,␣confirming␣communication␣works
␣␣␣␣"""
    return TextContent(
        type="text",
        text=f"Server␣received:␣{message}"
    )

def run_server():
    """Start␣the␣MCP␣server."""
    print("Starting␣Invoice␣MCP␣Server␣v1...")
    server.run()

if __name__ == "__main__":
    run_server()
```


© Dr. Segal Yoram - כל הזכויות שמורות

8.3 שלב 2: הוספת כלים

8.3.1 כלים לניהול חשבוניות

כלי יצירת חשבונית: Create Invoice Tool

```
# server/tools/invoice_tools.py
from mcp.server import Server
from mcp.types import TextContent, Tool
from pydantic import BaseModel
from typing import List
import json

class InvoiceItemInput(BaseModel):
    product_name: str
    quantity: int
    unit_price: float

class CreateInvoiceInput(BaseModel):
    customer_name: str
    customer_email: str
    items: List[InvoiceItemInput]

@server.tool(
    name="create_invoice",
    description="Create␣a␣new␣invoice␣for␣a␣customer",
    input_schema=CreateInvoiceInput.model_json_schema()
)
async def create_invoice(
    customer_name: str,
    customer_email: str,
    items: List[dict]
) -> TextContent:
    """
␣␣␣␣Create␣a␣new␣invoice␣with␣calculated␣totals.

␣␣␣␣This␣tool:
␣␣␣␣1.␣Validates␣the␣input
␣␣␣␣2.␣Calculates␣subtotal,␣tax,␣and␣total
␣␣␣␣3.␣Saves␣to␣database
␣␣␣␣4.␣Returns␣the␣created␣invoice␣details
␣␣␣␣"""
    # Calculate totals
    subtotal = sum(
        item["quantity"] * item["unit_price"]
        for item in items
    )
    tax_rate = 0.17  # 17% VAT
    tax_amount = subtotal * tax_rate
    total = subtotal + tax_amount

    # Save to database (simplified)
    invoice_id = db.save_invoice({
        "customer_name": customer_name,
```


© Dr. Segal Yoram - כל הזכויות שמורות

8.4 שלב 3: הוספת משאבים

Invoice List Resource :משאב רשימת חשבוניות

```
# server/resources/invoice_resources.py
from mcp.server import Server
from mcp.types import Resource, TextContent
import json

@server.list_resources()
async def list_invoice_resources() -> list[Resource]:
    """
␣␣␣␣List␣available␣invoice␣resources.

␣␣␣␣Returns␣URIs␣for:
␣␣␣␣-␣All␣invoices
␣␣␣␣-␣Pending␣invoices
␣␣␣␣-␣Paid␣invoices
␣␣␣␣-␣Monthly␣summary
␣␣␣␣"""
    return [
        Resource(
            uri="invoice://all",
            name="All␣Invoices",
            description="List␣of␣all␣invoices␣in␣the␣system",
            mime_type="application/json"
        ),
        Resource(
            uri="invoice://pending",
            name="Pending␣Invoices",
            description="Invoices␣awaiting␣payment",
            mime_type="application/json"
        ),
        Resource(
            uri="invoice://paid",
            name="Paid␣Invoices",
            description="Completed␣invoices",
            mime_type="application/json"
        ),
        Resource(
            uri="invoice://summary/monthly",
            name="Monthly␣Summary",
            description="Summary␣by␣month",
            mime_type="application/json"
        )
    ]

@server.read_resource()
async def read_invoice_resource(uri: str) -> TextContent:
    """
␣␣␣␣Read␣invoice␣data␣based␣on␣URI.
␣␣␣␣"""
```


© Dr. Segal Yoram - כל הזכויות שמורות

8.5 שלב 4: SDK וממשקים

SDK מלא לחשבוניות: Complete Invoice SDK

```
# sdk/invoice_sdk.py
from typing import List, Optional
from .models import Invoice, Customer, InvoiceItem
from .mcp_client import MCPClient
import json

class InvoiceSDK:
    """
␣␣␣␣SDK␣for␣invoice␣management.

␣␣␣␣Provides␣clean␣API␣for␣all␣invoice␣operations.
␣␣␣␣Can␣be␣used␣by␣CLI,␣GUI,␣or␣Web␣applications.
␣␣␣␣"""

    def __init__(self, server_command: str = "python␣server/main.py"):
        self._client = MCPClient(server_command)

    async def connect(self):
        """Connect␣to␣MCP␣server."""
        await self._client.connect()

    async def disconnect(self):
        """Disconnect␣from␣server."""
        await self._client.disconnect()

    # === Invoice Operations ===

    async def create_invoice(
        self,
        customer_name: str,
        customer_email: str,
        items: List[dict]
    ) -> Invoice:
        """Create␣a␣new␣invoice."""
        result = await self._client.call_tool(
            "create_invoice",
            {
                "customer_name": customer_name,
                "customer_email": customer_email,
                "items": items
            }
        )
        data = json.loads(result.text)
        return Invoice(**data)
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
    async def get_invoice(self, invoice_id: int) -> Optional[Invoice]:
        """Get␣invoice␣by␣ID."""
        result = await self._client.call_tool(
```

8.5.2 ממשק שורת פקודה

ממשק CLI מלא: Complete CLI Interface

```
# cli/main.py
import asyncio
import argparse
from sdk import InvoiceSDK

async def main():
    parser = argparse.ArgumentParser(
        description="Invoice␣Management␣CLI"
    )
    subparsers = parser.add_subparsers(dest="command")

    # Create command
    create = subparsers.add_parser("create", help="Create␣invoice")
    create.add_argument("--customer", required=True)
    create.add_argument("--email", required=True)
    create.add_argument("--items", required=True, help="JSON␣array")

    # List command
    list_cmd = subparsers.add_parser("list", help="List␣invoices")
    list_cmd.add_argument("--status", choices=["draft", "sent", "paid"])

    # Get command
    get_cmd = subparsers.add_parser("get", help="Get␣invoice")
    get_cmd.add_argument("id", type=int)

    # Summary command
    subparsers.add_parser("summary", help="Monthly␣summary")

    args = parser.parse_args()

    # Initialize SDK
    sdk = InvoiceSDK()
    await sdk.connect()

    try:
        if args.command == "create":
            import json
            items = json.loads(args.items)
            invoice = await sdk.create_invoice(
                args.customer, args.email, items
            )
            print(f"Created␣invoice␣#{invoice.id}")
            print(f"Total:␣{invoice.total:.2f}␣NIS")

        elif args.command == "list":
            invoices = await sdk.list_invoices(args.status)
            for inv in invoices:
                print(f"#{inv.id}:␣{inv.customer_name}␣-␣{inv.total:.2f}
```


© Dr. Segal Yoram - כל הזכויות שמורות

8.6 שלב 5: בדיקות

בדיקות יחידה לשרת: Server Unit Tests

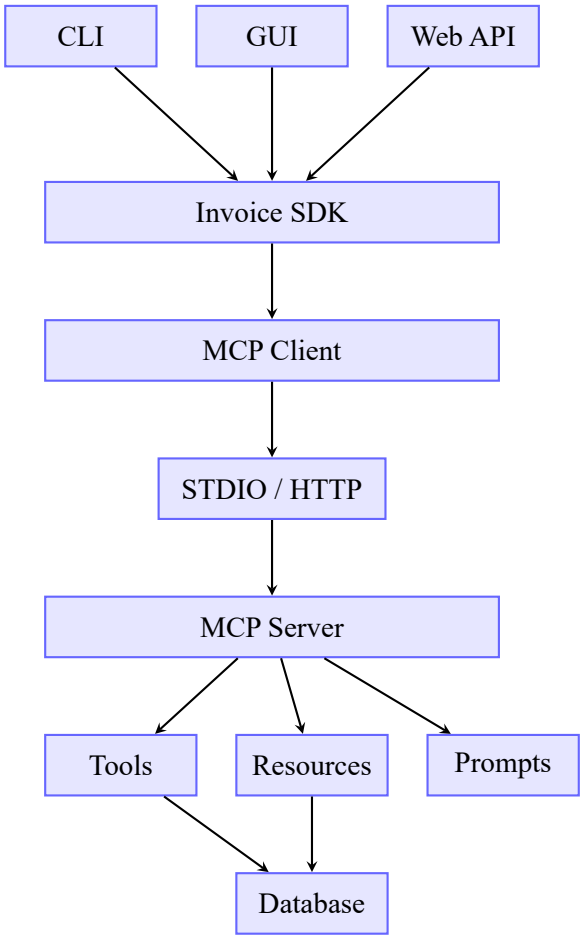
```
# tests/unit/test_invoice_tools.py
import pytest
from unittest.mock import Mock, patch
from server.tools.invoice_tools import create_invoice

@pytest.mark.asyncio
async def test_create_invoice_calculates_total():
    """Test␣invoice␣total␣calculation␣with␣VAT."""
    # Arrange
    with patch('server.tools.invoice_tools.db') as mock_db:
        mock_db.save_invoice.return_value = 1

        # Act
        result = await create_invoice(
            customer_name="Test␣Customer",
            customer_email="test@example.com",
            items=[
                {"product_name": "Widget", "quantity": 2, "unit_price":
                   100}
            ]
        )

        # Assert
        import json
        data = json.loads(result.text)
        assert data["success"] == True
        assert data["total"] == 234.0  # 200 + 17% VAT

@pytest.mark.asyncio
async def test_create_invoice_validates_items():
    """Test␣that␣empty␣items␣list␣is␣rejected."""
    result = await create_invoice(
        customer_name="Test",
        customer_email="test@example.com",
        items=[]
    )

    import json
    data = json.loads(result.text)
    assert data["success"] == False
    assert "error" in data

class TestSDKIntegration:
    """Integration␣tests␣for␣SDK␣with␣server."""
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
    @pytest.fixture
    async def sdk(self):
        sdk = InvoiceSDK()
```

8.7 סיכום הפרויקט

8.7.1 מה בנינו?



איור 5: ארכיטקטורה מלאה של מערכת החשבוניות. כל שכבה אחראית על משימה ספציפית, ואפשר להחליף כל רכיב בנפרד.

8.7.2 מה למדנו?

1. **ארכיטקטורה שכבתית** מאפשרת לבנות מערכות גמישות
2. **MCP** מספק שפה משותפת לסוכני AI
3. **SDK** מפריד לוגיקה מממשק
4. **בדיקות** מבטיחות איכות ואמינות
5. **מודולריות** מאפשרת פיתוח מקבילי והחלפת רכיבים

8.8 צעדים הבאים

לאחר השלמת הפרויקט הבסיסי, אפשר להרחיב:

- הוספת מנגנון אימות משתמשים

- חיבור ל-API של חשבונית ירוקה
- הוספת תמיכה במטבעות שונים
- יצירת PDF לחשבוניות
- הוספת דשבורד אנליטי
- אינטגרציה עם מערכות CRM

8.9 סיכום הספר

הגענו לסוף המסע. בספר זה למדנו:

1. **פרק 1** — הכרות עם עולם סוכני ה-AI
2. **פרק 2** — ארכיטקטורת שרת-לקוח והיחסים הדינמיים ב-MCP
3. **פרק 3** — שלושת הפרימיטיבים: Tools, Resources, Prompts
4. **פרק 4** — ארכיטקטורת תוכנה שכבתית ומודולרית
5. **פרק 5** — שכבת התקשורת: STDIO ו-HTTP/SSE
6. **פרק 6** — הפרדת SDK מממשק משתמש
7. **פרק 7** — בדיקות יחידה ו-TDD
8. **פרק 8** — יישום מעשי מלא

עכשיו יש לכם את הכלים לבנות מערכות מבוססות סוכני AI בצורה מקצועית, מודולרית, וניתנת לתחזוקה.

בהצלחה בפרויקטים שלכם!

כל הזכויות שמורות - Dr. Segal Yoram ©

נספח א: פיתוח מקבילי עם Git Worktrees

8.10 פתיחה: הבעיה העתיקה של עבודה במקביל

מאז ימי בניית הפירמידות במצרים העתיקה, האנושות התמודדה עם אתגר יסודי: כיצד לגרום למספר רב של עובדים לעבוד על אותו פרויקט בו-זמנית, מבלי שידרכו זה על רגליו של זה?

הפרעונים פתרו את הבעיה על ידי חלוקת העבודה לאזורים נפרדים — צוות אחד בנה את הפינה הצפונית-מזרחית, אחר את הדרומית-מערבית. כל צוות עבד באופן עצמאי, אך כולם תרמו לאותו מבנה גדול. כשצוות סיים את חלקו, הוא "מיזג" אותו לתוך המבנה הכללי.

אלפי שנים מאוחר יותר, מפתחי תוכנה מתמודדים עם בדיוק אותו אתגר. וכעת, כשסוכני AI נכנסים לתמונה, האתגר הופך למורכב עוד יותר: כיצד נאפשר למספר סוכנים חכמים לעבוד על אותו קוד בו-זמנית?

8.11 הפתרון: Git Worktrees

8.11.1 מה זה Worktree?

תארו לעצמכם ספרייה עם ספר יחיד. עכשיו תארו שאתם יכולים ליצור "העתקים מחוברים" של אותו ספר — כל העתק נמצא בחדר נפרד, אפשר לכתוב בו הערות שונות, אבל כל ההעתקים מחוברים באופן מסתורי לספר המקורי. כשאתם מחליטים שההערות שלכם מוכנות, הן "נספגות" לתוך הספר הראשי.

זה בדיוק מה ש-Git Worktree עושה: הוא יוצר תיקייה נפרדת שמכילה עותק של הקוד שלכם, על ענף (branch) משלו, אבל עדיין מחוברת לאותו מאגר (repository) מקורי.

הגדרה

Git Worktree הוא מנגנון שמאפשר לבדוק (checkout) מספר ענפים מאותו מאגר לתיקיות נפרדות בו-זמנית. כל Worktree הוא עצמאי לחלוטין מבחינת קבצים, אך משתף היסטוריה ומקור עם כל האחרים.

8.11.2 למה זה משנה את הכל?

עד היום, אם רצית לעבוד על שני פיצ'רים במקביל, היית צריך לבצע commit, להחליף ענף, לעבוד, להחליף חזרה... תהליך מתיש ומועד לטעויות.

עם Worktrees, אתה פשוט פותח שתי תיקיות — כל אחת על ענף אחר — ועובד בשתיהן במקביל. זה כמו להיות בשני מקומות בו-זמנית, בלי מכונת טלפורטציה.

8.12 הדפוס של Anthropic: סוכן לכל ענף

8.12.1 ההמלצה הרשמית

Anthropic, החברה שפיתחה את Claude, הבינה משהו חשוב: כשיש לך סוכן AI שעובד על קוד, אתה רוצה שהוא יעבוד בבידוד מלא. למה? כי אם שני סוכנים עובדים על אותם קבצים, הם ידרסו זה את זה.

הפתרון שלהם פשוט ואלגנטי: תן לכל סוכן Worktree משלו. זה נקרא **Branch-per-Agent Pattern** — דפוס "ענף לכל סוכן".

טבלה 13: דפוס "ענף לכל סוכן" — היתרונות

הסבר	יתרון
כל סוכן עובד על קבצים משלו. שינויים באחד לא משפיעים על השני	בידוד מלא
כל הסוכנים מחוברים לאותו מאגר, קל למזג את העבודה	היסטוריה משותפת
סוכנים יכולים לעבוד בו-זמנית בלי לחכות זה לזה	עבודה מקבילית אמיתית
Remote אחד, פקודות Git רגילות, אין צורך במערכות מיוחדות	ניהול פשוט

8.13 מדריך מעשי: כיצד להפעיל צוות סוכנים

8.13.1 שלב 1: יצירת Worktrees

נניח שיש לנו פרויקט בתיקייה /project. נרצה להפעיל שלושה סוכנים: אחד לפיצ'ר חדש, אחד לתיקון באג, ואחד לתיעוד.

יצירת Worktrees לשלושה סוכנים: Creating Worktrees

```
# Navigate to main project
cd /path/to/project

# Create worktree for Agent 1: Feature Development
git worktree add ../project-agent1-feature -b agent1/new-api

# Create worktree for Agent 2: Bug Fix
git worktree add ../project-agent2-bugfix -b agent2/fix-security

# Create worktree for Agent 3: Documentation
git worktree add ../project-agent3-docs -b agent3/update-readme

# Verify all worktrees exist
git worktree list
```

אחרי הרצת הפקודות, יהיו לנו ארבע תיקיות:

מבנה התיקיות: Directory Structure

```
/path/to/├──
 project/                    # Main (branch: main)├──
 project-agent1-feature/     # Agent 1 (branch: agent1/new-api)├──
 project-agent2-bugfix/      # Agent 2 (branch: agent2/fix-security)└──
 project-agent3-docs/        # Agent 3 (branch: agent3/update-readme)
```


כל הזכויות שמורות - Dr. Segal Yoram ©

8.13.2 שלב 2: הפעלת הסוכנים

כעת, בכל טרמינל נפרד, נפעיל סוכן Claude Code עם משימה ייעודית:

הפעלת סוכנים במקביל: Running Agents in Parallel

```
# Terminal 1: Agent 1 - Feature Development
cd ../project-agent1-feature
npm install  # Setup environment if needed
claude
> "Implement␣OAuth2␣authentication␣for␣the␣API"

# Terminal 2: Agent 2 - Bug Fix
cd ../project-agent2-bugfix
npm install
claude
> "Fix␣the␣SQL␣injection␣vulnerability␣in␣user␣login"

# Terminal 3: Agent 3 - Documentation
cd ../project-agent3-docs
claude
> "Update␣README␣with␣new␣API␣documentation"
```

8.13.3 שלב 3: מיזוג העבודה

כשסוכן מסיים את משימתו, ממזגים את הענף שלו ל-main:

מיזוג עבודת הסוכנים: Merging Agent Work

```
# Return to main project
cd /path/to/project

# Merge Agent 1's feature branch
git merge agent1/new-api -m "Feature:␣OAuth2␣authentication"

# Merge Agent 2's bugfix branch
git merge agent2/fix-security -m "Fix:␣SQL␣injection␣vulnerability"

# Merge Agent 3's docs branch
git merge agent3/update-readme -m "Docs:␣Updated␣API␣documentation"

# Push all changes to remote
git push origin main

# Clean up worktrees
git worktree remove ../project-agent1-feature
git worktree remove ../project-agent2-bugfix
git worktree remove ../project-agent3-docs
```

8.14 ניהול סשנים: כוחו של הזיכרון

8.14.1 שימור קונטקסט לכל סוכן

אחד היתרונות המפתיעים של Worktrees הוא שכל סוכן שומר את ההיסטוריה שלו. ב-Claude Code, הסשנים נשמרים לפי תיקייה, כך שאם סוכן מפסיק באמצע עבודה, אפשר להמשיך בדיוק מהנקודה שבה עצר:

המשך עבודה עם סשנים: Resuming Sessions

```
# Resume Agent 1's work from where it stopped
cd ../project-agent1-feature
claude --resume

# Or resume a specific named session
claude --resume feature-api-work
```

יתרה מזאת, פקודת resume/ מציגה סשנים מכל ה-Worktrees של אותו מאגר, כך שקל לנהל את כל הסוכנים ממקום אחד.

כל הזכויות שמורות - Dr. Segal Yoram ©

8.15 דוגמה מהעולם האמיתי: פיתוח מערכת חשבוניות

נחזור לדוגמת מערכת החשבוניות מפרק 8. נניח שיש לנו שלוש משימות גדולות:

1. הוספת תמיכה במטבעות שונים
2. תיקון באג בחישוב מע"מ
3. כתיבת בדיקות יחידה חדשות

במקום לעבוד על הכל ברצף, נפעיל שלושה סוכנים במקביל:

תרחיש מעשי: Practical Scenario

```
# Setup
git worktree add ../invoice-currencies -b feature/multi-currency
git worktree add ../invoice-vat-fix -b bugfix/vat-calculation
git worktree add ../invoice-tests -b test/new-unit-tests

# Agent 1: Currency support
cd ../invoice-currencies
claude "Add␣support␣for␣USD,␣EUR,␣and␣GBP␣currencies␣with␣real-time␣
   exchange␣rates"

# Agent 2: VAT bug fix
cd ../invoice-vat-fix
claude "Fix␣VAT␣calculation␣for␣exempt␣items␣and␣international␣orders"

# Agent 3: Test coverage
cd ../invoice-tests
claude "Write␣unit␣tests␣for␣InvoiceService␣with␣90%␣coverage␣target"
```

תוך שעה, שלושת הסוכנים יכולים להשלים משימות שהיו לוקחות יום שלם בעבודה רציפה.

8.16 טיפים מתקדמים

8.16.1 שמות ענפים תיאוריים

השתמשו בשמות שמתארים גם את הסוכן וגם את המשימה:

- agent1/feature-oauth — סוכן 1, פיצ'ר אימות
- agent2/bugfix-security — סוכן 2, תיקון אבטחה
- planner/architecture-review — סוכן מתכנן, סקירת ארכיטקטורה

8.16.2 סנכרון בין סוכנים

אם סוכן צריך לעדכן את הענף שלו מ-main:

סנכרון ענף: Branch Synchronization

```
# In agent's worktree
cd ../project-agent1-feature
git fetch origin
git rebase origin/main

# Or merge if you prefer
git merge origin/main
```

8.16.3 זיהוי קונפליקטים מוקדם

לפני מיזוג, בדקו קונפליקטים פוטנציאליים:

בדיקת קונפליקטים: Conflict Check

```
# Check if merge will have conflicts (dry run)
cd /path/to/project
git merge --no-commit --no-ff agent1/new-api
git merge --abort  # Undo the test merge

# If conflicts exist, resolve them in the worktree first
```

8.17 סיכום: העתיד כבר כאן

לפני אלפי שנים, הפרעונים הצליחו לבנות פירמידות על ידי חלוקת העבודה לאזורים נפרדים. היום, אנחנו משתמשים באותו עיקרון — רק שבמקום פועלים אנושיים, יש לנו סוכני AI.

Git Worktrees עם דפוס "ענף לכל סוכן" הוא לא רק טריק טכני — זו פרדיגמה חדשה לפיתוח תוכנה. במקום מפתח אחד שעובד על משימה אחת, יש לנו "צוות" של סוכנים שעובדים במקביל, כל אחד בתחום שלו, וכולם תורמים לאותו פרויקט.

זה לא רק מהיר יותר — זה משנה את עצם הדרך שבה אנחנו חושבים על פיתוח תוכנה.

העתיד של פיתוח תוכנה הוא עבודה במקביל עם סוכנים חכמים.

כל הזכויות שמורות - Dr. Segal Yoram ©

8.18 מקורות בעברית

8.19 English References